



西安电子科技大学
XIDIAN UNIVERSITY

第二章 计算机系统中的数据表示

西安电子科技大学

人工智能学院

林杰



一、数值数据的表示

1. 数据格式
2. 定点数表示
3. 浮点数表示

二、非数值数据的表示

1. 字符码
2. 汉字编码
3. Unicode编码

三、数据信息的校验

1. 码距与数据校验
2. 奇偶校验
3. 海明校验
4. 循环冗余校验



- 计算机内部流动的信息可以分为两大类
 - 数据信息，是计算机加工处理的对象。
 - 控制信息，用于控制数据信息的加工处理。
- 数据信息的表示直接影响运算器的设计，进而也可能影响计算机的结构和性能。



在设计和选择计算机内的数据表示方式时，一般需要综合考虑以下因素：

- (1) 数据的类型：一般要支持数值数据和非数值数据，前者如小数、整数、实数等，后者如ASCII码和汉字等。
- (2) 表示的范围和精度：通过选择适当的数据类型与字长来实现。
- (3) 存储和处理的代价：应尽量使设计出的数据格式易于表示、存储和处理；易于设计处理数据的硬件，如运算器设计等需要综合考虑性能需求和硬件开销。
- (4) 软件的可移植性：从保护用户软件投资的角度看，应使设计的数据格式在满足应用需求的前提下，符合相应的规范，方便软件在不同计算机之间的移植。

文字、图、表、声音、视频等各种媒体信息

最终用户角度

输入设备

输出设备

二进制编码表示的各种数据

图、树、链表等结构化数据描述

高级语言程序员角度

指令系统能识别的基本类型数据

低级语言程序员和
硬件系统设计者角度

数值型数据

非数值型数据

二进制数

二进制编码的
十进制数

逻辑数据

编码字符，如西文和汉字

整数（定点数）

实数（浮点数）

无符号整数

带符号整数

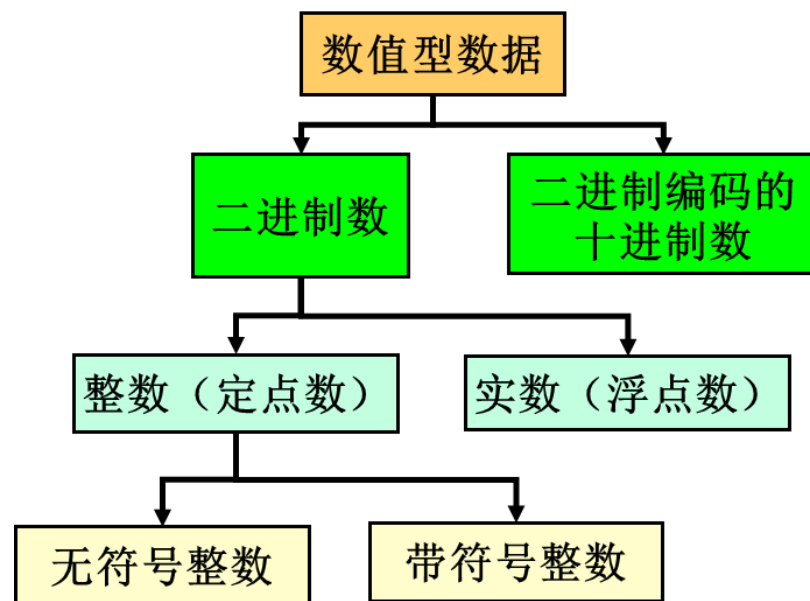




一、数值数据的表示

1. 数据格式

- 数据格式是指使用二进制编码表示实际数据的结构形式。
- 二进制编码的优势：
- 有两个稳定状态的物理器件容易制造
- 编码和运算规则都很简单。可用开关电路实现，简单易行。
- 1和0与逻辑“真”和“假”相对应，为逻辑运算和判断提供了便利的条件，特别是能通过逻辑门电路方便地实现算术运算。



根据**小数点的位置**可以分为**定点数**和**浮点数**两类。
根据**是否有符号位**可以分为**无符号数**和**有符号数**两类。



一、数值数据的表示

无符号数

- 无符号数总是正整数，在机器数中没有符号位；
- 一般在全部是正数运算且不出现负值结果的情况下使用。例如地址运算，编号表示等。

有符号数

- 用数字化的“0”表示“正”，“1”表示“负”，符号放在有效数字的最前面。

如果使用的位数相同，无符号数能表示的最大值大于带符号整数的最大值。8位无符号整数最大是255（1111 1111），8位带符号整数最大为127（0111 1111）。

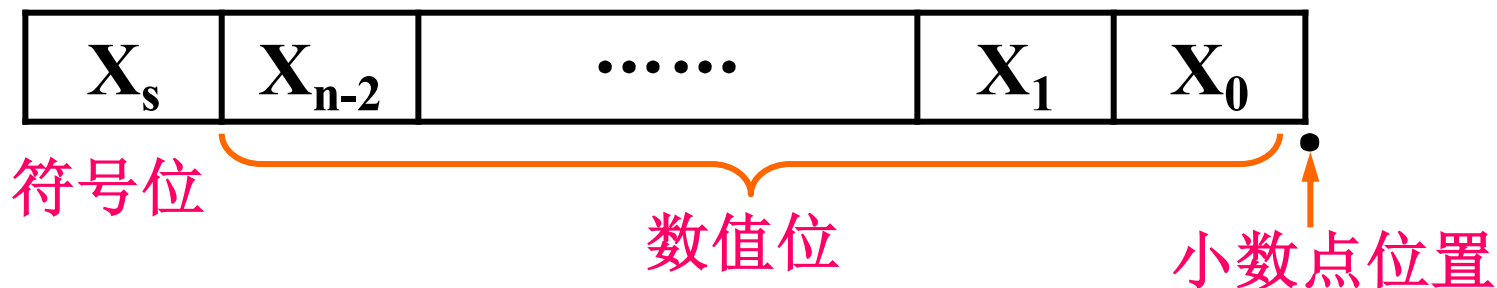


一、数值数据的表示

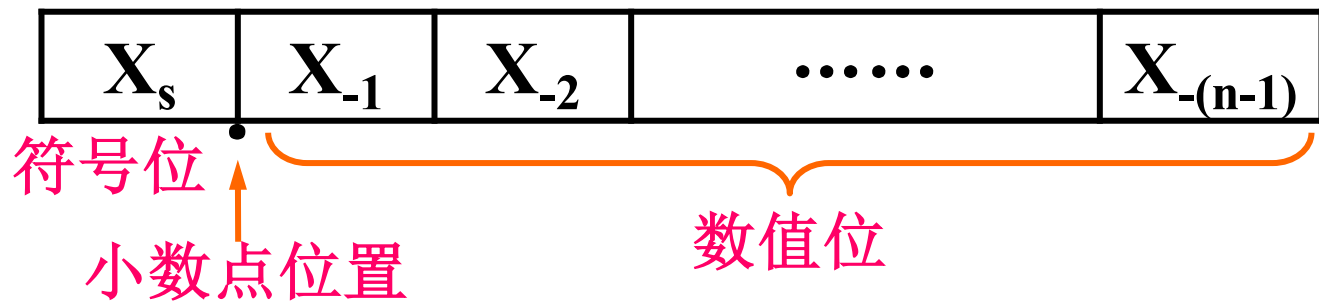
• 定点与浮点表示

– 计算机中定点数只定义为纯整数或纯小数形式；

带符号定点整数格式：



带符号定点小数格式：





一、数值数据的表示

浮点表示

- 对于任意一个二进制数 X ，可以表示成如下形式：

$$X=(-1)^s \times M \times R^E$$

- S 取值为0或1，用来决定数 X 的符号
- M 是一个二进制定点小数，称为数 X 的尾数(mantissa)
- E 是一个二进制定点整数，称为数 X 的阶或指数(exponent)
- R 是基数(radix, base)，可以取值为2、4、16等。

在基数 R 一定的情况下

- 尾数 M 的位数反映数 X 的有效位数，它决定了数的表示精度，有效位数越多，表示精度就越高；
- 指数 E 的位数决定数 X 的表示范围，其值确定了小数点的位置。



一、数值数据的表示

- 定点/浮点表示解决了小数点的表示问题。
- **符号数字化**：将数的符号用0和1表示的处理方式。
- 一般规定0表示正号，1表示负号。
- **定点数的编码表示：原码、补码、反码、移码**
- 通常将数值数据在计算机内部编码表示的数称为**机器数**。
- 机器数真正的值(即现实世界中带有正负号的数)称为机器数的**真值**。
- 如-10(-1010B)用8位补码表示为11110110，说明机器数11110110B(F6H或0xF6)的真值是-10，或者说，-10的机器数是11110110B(F6H或0xF6)。



一、数值数据的表示

- 数值数据表示的三要素

- 进位计数制
- 定、浮点表示
- 如何用二进制编码

例如，机器数 01011001 的值是多少？

- ① 进位计数制：十进制、二进制、十六进制、八进制数
- ② 定/浮点表示（解决小数点问题）
 - 定点整数、定点小数
 - 浮点数（可用一个定点小数和一个定点整数来表示）
- ③ 定点数的编码（解决正负号问题）：原码、补码、反码、移码₁



1.1 计算机常用各种进制数的表示

- **基数**：计数制中用到的数码的个数，用**R**表示。
- **位权**：以基数为底的指数，指数的幂是数位的序号。
- 对一个数S，其基数为**R**，则：

$$\begin{aligned}(S)_R &= \pm(K_{n-1}K_{n-2} \dots K_2K_1K_0.K_{-1}K_{-2} \dots K_{-m}) \\&= \pm(K_{n-1}R^{n-1} + K_{n-2}R^{n-2} + \dots + K_1R^1 + K_0R^0 + K_{-1}R^{-1} + \dots + K_{-m}R^{-m}) \\&= \pm \sum_{i=-m}^{n-1} K_i R^i\end{aligned}$$



一、数值数据的表示

计算机常用各种进制数的表示

进位制	二进制	八进制	十进制	十六进制
规则	逢二进一	逢八进一	逢十进一	逢十六进一
基数	$R=2$	$R=8$	$R=10$	$R=16$
基本符号	0,1	0,1,2,...,7	0,1,2,...,9	0,1,...,9,A,...,F
权	2^i	8^i	10^i	16^i
形式表示	B	O/Q	D	H



一、数值数据的表示

【例2.1】 将二进制数 $(11001.01)_2$ 、八进制数 $(216.3)_8$ 、十六进制数 $(7A.C)_{16}$ 转换成十进制数。

【解】 “按权展开”

$$(11001.01)_2$$

$$=(1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2})_{10}$$

$$=(25.25)_{10}$$

$$(216.3)_8$$

$$=(2 \times 8^2 + 1 \times 8^1 + 6 \times 8^0 + 3 \times 8^{-1})_{10}$$

$$=(142.375)_{10}$$

$$(7A.C)_{16}$$

$$=(7 \times 16^1 + 10 \times 16^0 + 12 \times 16^{-1})_{10}$$

$$=(122.75)_{10}$$



1.2 十进制转换为二进制数

• 任一十进制数 N ， $N=N_{\text{整}}+N_{\text{小}}$ 。将这两部分分开转换

①整数部分的转换：采用“除2求余法”：连续用2除，求得余数（1或0）分别为 K_0 、 K_1 、 K_2 、...，直到商为0，所有余数排列 $K_{n-1}K_{n-2}...K_2K_1K_0$ 即为所转换的二进制整数部分。

②小数部分的转换：采用“乘2取整法”：连续用2乘，依次求得各整数位（0或1） K_{-1} 、 K_{-2} 、...、 K_{-m} ，直到乘积的小数部分为0。在小数转换过程中，出现 F_i 恒不为0时，可按精度要求确定二进制小数的位数。



一、数值数据的表示

【例2.2】将十进制数730.8125转换成二进制数、八进制数。

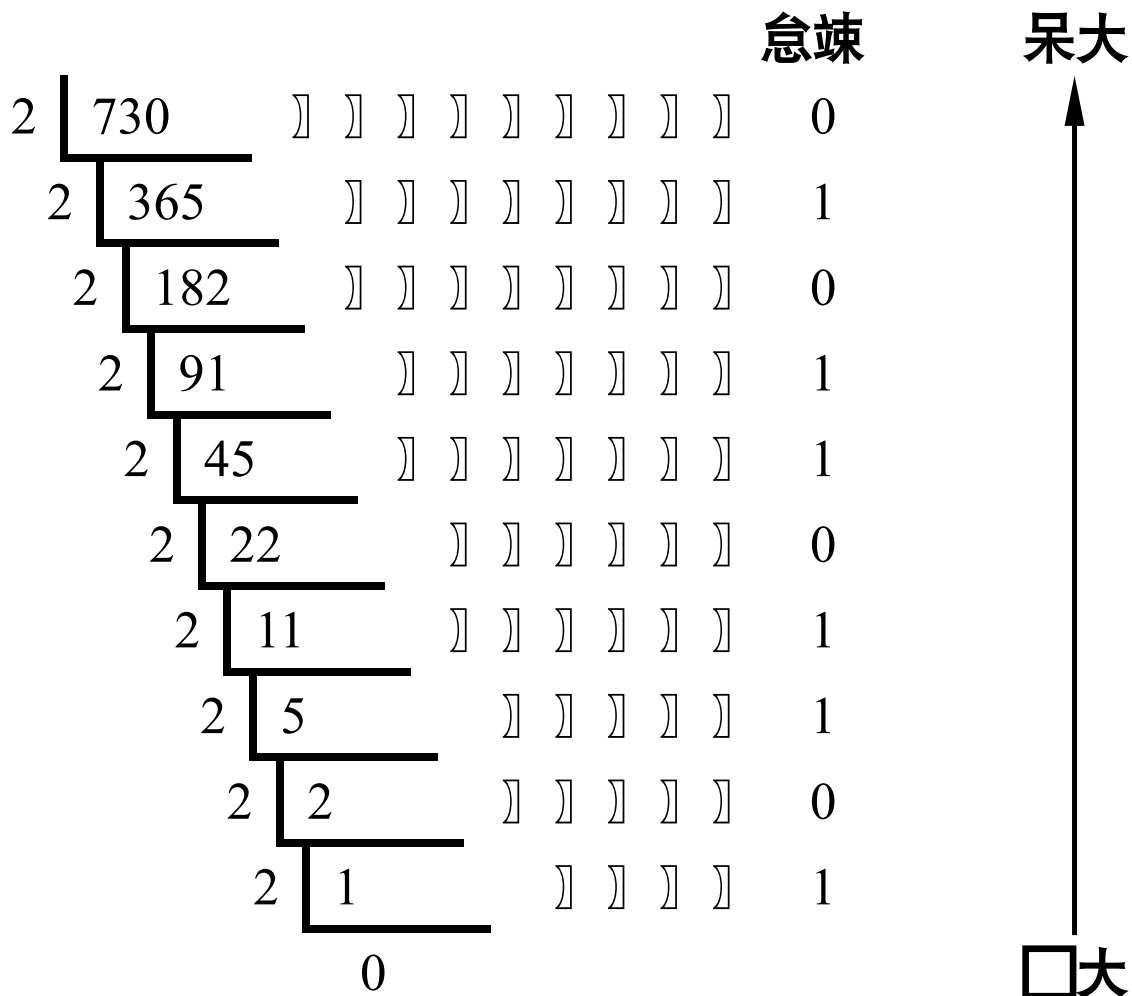
【解】① 整数部分的转换：“除基取余，先低后高”

							总	大
8	730	]	]	]	]	]	2	
8	91	]	]	]	]	]	3	
8	11	]	]	]	]	]	3	
8	1	]	]	]	]		1	
	0	]	]	]	]		0	
								大

因此， $(730)_{10} = (1332)_8$



一、数值数据的表示



因此, $(730)_{10} = (1011011010)_2$



一、数值数据的表示

【例2.2】将十进制数730.8125转换成二进制数、八进制数。

【解】 ② 小数部分的转换：“乘基取整，先高后低”

将十进制数 $(0.8125)_{10}$ 转换为八进制数：

$$0.8125 \times 8 = 6.5 \quad \text{整数部分} = 6 \quad (\text{高位})$$

$$0.5 \times 8 = 4.0 \quad \text{整数部分} = 4 \quad (\text{低位})$$

$$\text{因此, } (0.8125)_{10} = (0.64)_8$$

$$\therefore (730.8125)_{10} = (1332.64)_8$$



一、数值数据的表示

【例2.2】将十进制数730.8125转换成二进制数、八进制数。

【解】② 小数部分的转换：“乘基取整，先高后低”

将十进制数 $(0.8125)_{10}$ 转换为二进制数：

$$0.8125 \times 2 = 1.625$$

整数部分=1 （高位）

$$0.625 \times 2 = 1.25$$

整数部分=1

$$0.25 \times 2 = 0.5$$

整数部分=0

$$0.5 \times 2 = 1.0$$

整数部分=1 （低位）

因此， $(0.8125)_{10} = (0.1101)_2$

$$\therefore (730.8125)_{10} = (1011011010.1101)_2$$



1.3 十进制数转换为八进制数、十六进制数

- 将十进制数转换为八进制数、十六进制数时，使用的方法与十进制数转换成二进制数的方法基本相同，只是求整数部分时是用商除以8或16，取其余数；小数部分改用乘以8或16，取其整数即可。



1.4 二进制数与八进制、十六进制数间的转换

- 二进制转化成八(十六)进制

- 整数部分：从右向左按三(四)位分组，不足补零
- 小数部分：从左向右按三(四)位分组，不足补零

【例2.3】

(001 011 010 110.101 011 100)₂

1 3 2 6 5 3 4

(001 011 010 110.101 011 100)₂ = (1326.534.)₈

(0101 1101.0101 1010)₂

5 D 5 A

(0101 1101.0101 1010)₂ = (5D.5A)₁₆



八进制、十六进制数与二进制数间的转换

- 八(十六)进制转化成二进制
 - 一位八进制数对应三位二进制数
 - 一位十六进制数对应四位二进制数
- 【例2.4】将下列数从八(十六)进制转化成二进制

$(247.63)_8$

$$= (\underline{010} \ \underline{100} \ \underline{111}.\underline{110} \ \underline{011})_2$$

$(F5A.6B)_{16}$

$$= (\underline{1111} \ \underline{0101} \ \underline{1010} \ .\underline{0110} \ \underline{1011})_2$$



一、数值数据的表示

1. 数据格式
2. 定点数表示
3. 浮点数表示
4. 计算机中的数据类型

二、非数值数据的表示

1. 字符码
2. 汉字编码
3. Unicode编码

三、数据信息的校验

1. 码距与数据校验
2. 奇偶校验
3. 海明校验
4. 循环冗余校验



一、数值数据的表示—2.定点数

2. 定点数

假设机器数 X 的真值 X_T 为

$$X_T = \pm X'_1 X'_2 \dots X'_n \quad (\text{当} X \text{为定点整数时})$$

$$X_T = \pm 0.X'_1 X'_2 \dots X'_n \quad (\text{当} X \text{为定点小数时})$$

对 X_T 用 $n+1$ 位二进制数编码后，机器数 X 表示为

$$X = X_0 X_1 X_2 \dots X_n$$

- 机器数 X 的位数为 $n+1$ ，其中，第一位 X_0 是数的符号位，后面 n 位 $X_1 X_2 \dots X_n$ 是数值部分。

数值数据在计算机内部的编码问题，实际上就是机器数 X 的各位 X_i 的取值与真值 X_T 的关系问题。



一、数值数据的表示—2.定点数

2.1 原码表示（“符号-数值”表示法）

- 最高位是符号位

– 0:正数

– 1:负数

- 数值位:真值的绝对值

- 对于定点整数:

若 $X = +X_1X_2...X_n$, 则 $[X]_{\text{原}} = 0 X_1X_2...X_n$;

若 $X = -X_1X_2...X_n$, 则 $[X]_{\text{原}} = 1 X_1X_2...X_n$ 。

- 对于定点小数:

若 $X = +0.X_1X_2...X_n$, 则 $[X]_{\text{原}} = 0.X_1X_2...X_n$;

若 $X = -0.X_1X_2...X_n$, 则 $[X]_{\text{原}} = 1.X_1X_2...X_n$ 。

- 二进制 $n+1$ 位表示的定点整数的原码定义如下:

$$[X]_{\text{原}} = \begin{cases} X & 0 \leq X < 2^n \\ 2^n - X = 2^n + |X| & -2^n < X \leq 0 \end{cases}$$

X 为真值, n 为整数数值位的位数。

- 二进制 $n+1$ 位表示的定点小数的原码定义如下:

$$[X]_{\text{原}} = \begin{cases} X & 0 \leq X < 1 \\ 1 - X = 1 + |X| & -1 < X \leq 0 \end{cases}$$



一、数值数据的表示—2.定点数

【例2.6】若机器字长为8，则+35、-35、+0.8125、-0.8125的原码表示。

$$[+35]_{\text{原}} = (00100011)_2$$

$$\begin{aligned} [-35]_{\text{原}} &= 2^7 - (-35) \\ &= (10000000)_2 + (00100011)_2 \\ &= (10100011)_2 \end{aligned}$$

$$[+0.8125]_{\text{原}} = (0.1101000)_2$$

$$\begin{aligned} [-0.8125]_{\text{原}} &= 1 - (-0.8125) \\ &= (1.0000000)_2 + (0.1101000)_2 \\ &= (1.1101000)_2 \end{aligned}$$

2^{-1}	0.5
2^{-2}	0.25
2^{-3}	0.125
2^{-4}	0.0625
2^{-5}	0.03125

2^0	1
2^1	2
2^2	4
2^3	8
2^4	16
2^5	32
2^6	64
2^7	128
2^8	256
2^9	512
2^{10}	1024



一、数值数据的表示—2.定点数

例 2.7 已知 $[x]_{\text{原}} = 1.0011$ 求 x



解： 由定义得 $x = 1 - [x]_{\text{原}} = 1 - 1.0011 = -0.0011$

例 2.8 已知 $[x]_{\text{原}} = 1,1100$ 求 x



解： 由定义得 $x = 2^4 - [x]_{\text{原}} = 10000 - 1,1100 = -1100$

例 2.9 求 $x = 0$ 的原码

解： 设 $x = + 0.0000$

$$[+ 0.0000]_{\text{原}} = 0.0000$$

$x = - 0.0000$

$$[-0.0000]_{\text{原}} = 1.0000$$

同理，对于整数

$$[+ 0]_{\text{原}} = 0,0000$$

$$[-0]_{\text{原}} = 1,0000$$

$\therefore [+ 0]_{\text{原}} \neq [-0]_{\text{原}}$



一、数值数据的表示—2.定点数

表示范围（机器字长为 $n+1$ ）：定点小数

2^0 2^{-1} 2^{-2} 2^{-3} ... $2^{-(n-1)}$ 2^{-n}

最大正数	0	1	1	1	...	1	1
最小正数	0	0	0	0	...	0	1

最大正数对应的真值为 $2^{-1}+2^{-2}+...+2^{-n}$

因为 $1.00...00$ $2^0=1$

$-0.00...01$ 2^{-n}

$0.11...11$ $1-2^{-n}$

最大正数对应的真值为 $1-2^{-n}$

最小正数对应的真值为 2^{-n}

原码表示时，正数和负数的范围对称，绝对值最大的负数为 $-(1-2^{-n})$ ，绝对值最小的负数为 -2^{-n}



一、数值数据的表示—2.定点数

表示范围（机器字长为 $n+1$ ）：定点**整数**

2^n 2^{n-1} 2^{n-2} 2^{n-3} ... 2^1 2^0

最大负数	0	1	1	1	...	1	1
最小正数	0	0	0	0	...	0	1

最大正数对应的真值为 $2^{n-1}+2^{n-2}+...+2^0=2^n-1$

最小正数对应的真值为 $2^0=1$

绝对值最大负数对应的真值为 $-(2^{n-1}+2^{n-2}+...+2^0)=-(2^n-1)$



一、数值数据的表示—2.定点数

原码的性质：

— “0”不唯一。 $[+0]_{\text{原}} = 0\ 00\dots 0$, $[-0]_{\text{原}} = 1\ 00\dots 0$

— 表示范围（机器字长为 $n+1$ ）：**对称**

• 定点小数： $-(1-2^{-n}) \leq X \leq +(1-2^{-n})$

即 $1.111\dots 11$ (n 个1) $\sim$ $0.11\dots 11$ (n 个1)

• 定点整数： $-(2^n-1) \leq X \leq +(2^n-1)$

即 $1111\dots 11$ (n 个1) $\sim$ $011\dots 11$ (n 个1)

— 负数的原码大于正数的原码。



一、数值数据的表示—2.定点数

原码的优缺点：

- 优点：
 - 简单、直观，机器数和真值间的相互转换很容易；
 - 实现乘、除运算的规则简单。
- 缺点：
 - 实现加、减运算的规则较复杂；
 - “0” 的表示不唯一。

要求	数1	数2	实际操作	结果符号
加法	正	正	加	正
加法	正	负	减	可正可负
加法	负	正	减	可正可负
加法	负	负	加	负



2.2 补码（2-补码）表示

• 有模运算

- 在模运算系统中，若A、B、M满足下列关系：

$A=B+K\times M$ (为整数)，则记为： $A\equiv B(\text{mod } M)$ 。称B和A为模M同余。

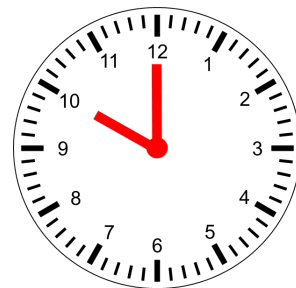
- 在一个模运算系统中，一个数与它除以“模”后得到的余数

是等价的

假定现在钟表时针指向10点，要将它拨向6点，有以下两种拨法。

(1)倒拨4格： $10-4=6$ 。

(2)顺拨8格： $10+8=18\equiv 6(\text{mod } 12)$ 。



所以在模12系统中， $10-4=10+(12-4)\equiv 10+8(\text{mod } 12)$ 。即： $-4\equiv 8(\text{mod } 12)$ 。

我们称8是-4对模12的补码。同样有 $-3\equiv 9(\text{mod } 12)$ ； $-5\equiv 7(\text{mod } 12)$ 等。



一、数值数据的表示—2.定点数

结论：对于某一确定的模，某数A减去小于模的另一数B，可以用A加上-B的补码来代替。

- 例：假定在钟表上只能顺拨时针，则如何用顺拨的方式实现将10点倒拨4格，拨动后钟表上是几点？
- 解：钟表是一个模运算系统，其模为12。因为 $10-4 \equiv 10+(12-4) \equiv 10+8 \equiv 6 \pmod{12}$ ，所以，可从10点顺拨8格来实现倒拨4格，最后拨到6点。
- 例：假定算盘只有4挡，且只能做加法，则如何用该算盘计算9828-1928的结果？
- 解：这个算盘是一个“4位十进制数”模运算系统，其模为 10^4 。

$$9828-1928 \equiv 9828+(10^4-1928) \equiv 9828+8072 \equiv 7900 \pmod{10^4}$$

可用9828加8072(-1928的补码)来实现9828减1928的功能。



一、数值数据的表示—2.定点数

对于 $n+1$ 位的定点数，其模的概念。

- 需要搞清楚每位二进制数的位权。
- 当这个二进制数为全1，再加上“1”（即在二进制的最低位上加1）时，向更高位的进位就是模，定点小数和定点整数的模分别由图（a）和（b）所示。

2^1	2^0	2^{-1}	2^{-2}	...	$2^{-(n-1)}$	2^{-n}
	1	1	1	...	1	1
						1
+						
1	0	0	0	...	0	0
模						

(a) 定点小数的模 (2)

2^{n+1}	2^n	2^{n-1}	2^{n-2}	...	2^1	2^0
	1	1	1	...	1	1
						1
+						
1	0	0	0	...	0	0
模						

(b) 定点整数的模 (2^{n+1})



一、数值数据的表示—2.定点数

补码（机器字长为 $n+1$ ）：

设模为 M ，一个数 X 补码的一般定义为：

$$[X]_{\text{补}} = M + X \pmod{M}$$

- 若 $X > 0$ ，则模 M 作为超出部分被舍去， $[X]_{\text{补}} = X$ ，因而正数的补码就是其本身；
- 若 $X < 0$ ，则 $[X]_{\text{补}}$ 就是以 M 为模的补数，等于模与该负数绝对值之差（ $M - |X|$ ）。
- 定点整数的补码定义：

$$[X]_{\text{补}} = \begin{cases} 0, X & 0 \leq X < 2^n \\ 2^{n+1} + X & -2^n \leq X < 0 \end{cases} \pmod{2^{n+1}}$$

- 定点小数的补码定义：

$$[X]_{\text{补}} = \begin{cases} X & 0 \leq X < 1 \\ 2 + X & -1 \leq X < 0 \end{cases} \pmod{2}$$



一、数值数据的表示—2.定点数

由定义可以求出补码的表示形式:

- 对于定点整数:

若 $X = +X_1X_2...X_n$, 则 $[X]_{\text{补}} = 0X_1X_2...X_n$;

若 $X = -X_1X_2...X_n$, 则 $[X]_{\text{补}} = 1\overline{X_1}\overline{X_2}... \overline{X_n} + 1$



- 对于定点小数:

若 $X = +0.X_1X_2...X_n$, 则 $[X]_{\text{补}} = 0.X_1X_2...X_n$;

若 $X = -0.X_1X_2...X_n$, 则 $[X]_{\text{补}} = 1.\overline{X_1}\overline{X_2}... \overline{X_n} + 0.00...1$



一、数值数据的表示—2.定点数

“按位取反，末位加一”的等效实现方式：

数值部分自低位向高位搜索，第一个1及其以右的各位0保持不变，以左的各高位按位取反。

$X \mid XX, + \mid ++ \gg \gg$ 垒阉竦啡

⌘ 赅大炼镰

$\bar{X} \mid \bar{X} \bar{X} +, \mid , , \gg \gg$ 垒阉竦啡赅大炼镰

; , $\gg \gg$ 貌大进崩

$\boxed{\bar{X} \mid \bar{X} \bar{X}} \boxed{, + \mid ++} \gg \gg$ 垒阉竦啡赅大炼镰 X 貌大进崩箔令 $\square \square$ 鳢

赅大炼镰 ⊙ ⊙ 莩粮

$\boxed{X \mid XX} \boxed{, + \mid ++} \gg \gg$ 垒阉竦啡



一、数值数据的表示—2.定点数

【例2.11】若机器字长为8，则

$$[+35]_{\text{补}} = (0010\ 0011)_2$$

$$[-35]_{\text{补}} = (1101\ 1101)_2$$

$$[+0.8125]_{\text{补}} = (0.1101000)_2$$

$$[-0.8125]_{\text{补}} = (1.0011000)_2$$

2^{-1}	0.5
2^{-2}	0.25
2^{-3}	0.125
2^{-4}	0.0625
2^{-5}	0.03125

2^0	1
2^1	2
2^2	4
2^3	8
2^4	16
2^5	32
2^6	64
2^7	128
2^8	256
2^9	512
2^{10}	1024

$$[+35]_{\text{原}} = (00100011)_2$$

$$[+0.8125]_{\text{原}} = (0.1101000)_2$$



补码的性质

1) 补码的符号位

用补码表示的数，若其最高位为“0”，则此数为正；若其最高位为“1”，则此数为负。

2) 补码中0的表示：

由补码定义， $[X]_{\text{补}} = M + X \pmod{M}$

即 $[0]_{\text{补}} = 0 = 00\dots 0$

∴ 0的补码是唯一的。



一、数值数据的表示—2.定点数

3) 补码的表示范围:

正数的补码与原码相同，因此小数和整数补码的最大正数和最小正数与原码相同。但是原码和补码表示的负数部分不同。

原码表示的绝对 值最大的负数	2^0	2^{-1}	2^{-2}	...	$2^{-(n-1)}$	2^{-n}
	1	1	1	...	1	1

补码表示的绝对 值最大的负数	2^0	2^{-1}	2^{-2}	...	$2^{-(n-1)}$	2^{-n}
	1	0	0	...	0	0

原码和补码表示的绝对值最大的负数（定点小数）

原码表示的绝对值最大的负数，其真值为 $-(1 - 2^{-n})$

补码表示的绝对值最大的负数，其真值为 -1 。



一、数值数据的表示—2.定点数

3) 补码的表示范围:

原码表示的绝对 值最大的负数	2^n	2^{n-1}	2^{n-2}	...	2^1	2^0
	1	1	1	...	1	1
补码表示的绝对 值最大的负数	2^n	2^{n-1}	2^{n-2}	...	2^1	2^0
	1	0	0	...	0	0

原码和补码表示的绝对值最大的负数（定点整数）

原码表示的绝对值最大的负数，其真值为 $-(2^n-1)$;

补码表示的绝对值最大的负数，其真值为 -2^n 。

在原码中表示“-0”的编码 $10...0$ 则用来表示值定点整数 -2^n 和定点小数 -1 （最负的数），最高位的1有两个含义，既代表符号，又代表这一位的位权。对于定点整数，其值为 -2^n ，对于定点小数，其值为 $-2^0=-1$ 。



3) 补码的表示范围:

假设**机器字长为 $n+1$** ，用补码表示的**定点小数**，其表示范围为：

$$-1 \leq X \leq +(1-2^{-n})$$

即100 ...0 (n个0) ~ 011...1 (n个1)

用补码表示的**定点整数**，其表示范围为：

$$-2^n \leq X \leq +(2^n-1)$$

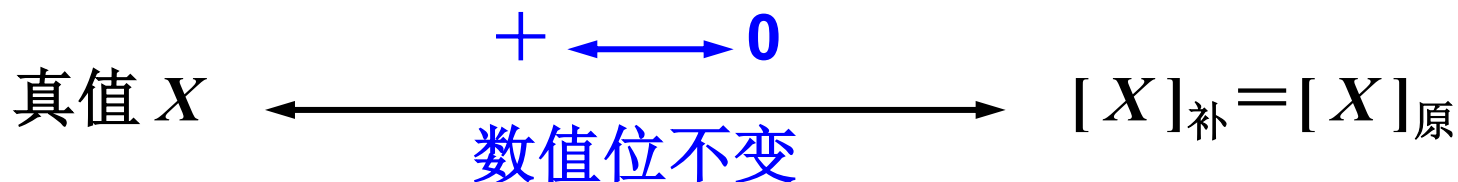
即100 ...0 (n个0) ~ 011...1 (n个1)

4) 负数的补码值大于正数的补码值

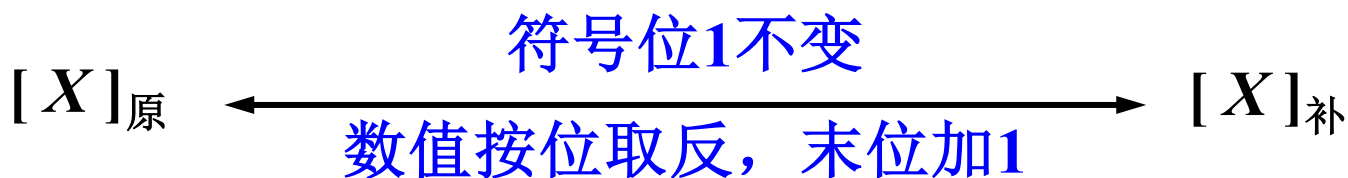
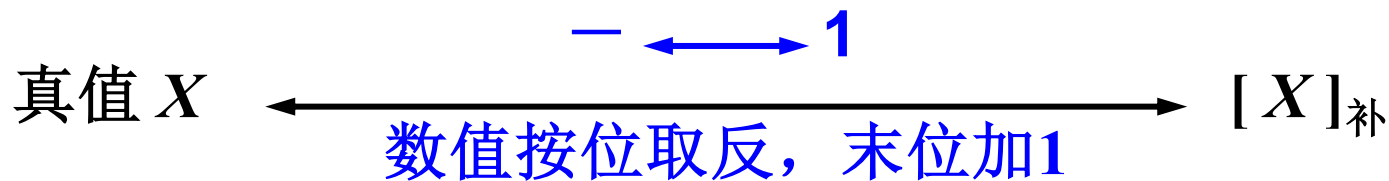


5) 补码与真值、原码之间的相互转换

当 $X \geq 0$ 时, $X = [X]_{\text{原}} = [X]_{\text{补}}$



当 $X < 0$ 时,





【例2.12】 假设机器字长为8， 已知

$$X_1 = +0.1011001,$$

$$X_2 = 0,$$

$$X_3 = -0.1101100,$$

求其原码及补码。

解： $[X_1]_{\text{补}} = [X_1]_{\text{原}} = X_1 = (0.1011001)_2$

$$[X_2]_{\text{补}} = (0.000\ 0000)_2$$

$$[X_2]_{\text{原}} = (0.000\ 0000)_2 = (1.000\ 0000)_2$$

$$[X_3]_{\text{原}} = (1.1101100)_2$$

$$[X_3]_{\text{补}} = (1.0010\mathbf{100})_2$$



【例2.13】 已知

$$[X_1]_{\text{补}} = 0.101\ 0110,$$

$$[X_2]_{\text{补}} = 1.110\ 0101,$$

$$[X_3]_{\text{补}} = 1.000\ 0000, \text{ 求其真值及原码。}$$

解: $X_1 = 0.101\ 0110$

$$[X_1]_{\text{原}} = 0.101\ 0110$$

$$X_2 = -0.001\ 1011$$

$$[X_2]_{\text{原}} = 1.001\ 1011$$

$$X_3 = -1$$

$$[X_3]_{\text{原}} \text{ 不存在}$$



2.3 反码表示（1-补码）

- 设机器字长为 $n+1$ 位，定点整数的反码定义：

$$[X]_{\text{反}} = 2^{n+1} - 1 + X \pmod{2^{n+1} - 1}$$

$$[X]_{\text{反}} = \begin{cases} 0, & X \\ (2^{n+1} - 1) + X & -2^n < X \leq 0 \end{cases} \quad \text{Mod } (2^{n+1} - 1)$$

- 设机器字长为 $n+1$ 位，定点小数的反码定义：

$$[X]_{\text{反}} = 2 - 2^{-n} + X \pmod{(2 - 2^{-n})}$$

$$[X]_{\text{反}} = \begin{cases} X & 0 \leq X < 1 \\ (2 - 2^{-n}) + X & -1 < X \leq 0 \end{cases} \quad \text{Mod}(2 - 2^{-n})$$



一、数值数据的表示—2.定点数

由定义可以求出反码的表示形式：

- 对于定点整数：

若 $X = +X_1X_2...X_n$ ，则 $[X]_{\text{反}} = 0, X_1X_2...X_n$ ；

若 $X = -X_1X_2...X_n$ ，则 $[X]_{\text{反}} = 1\overline{X_1}\overline{X_2}... \overline{X_n}$

- 对于定点小数：

若 $X = +0.X_1X_2...X_n$ ，则 $[X]_{\text{反}} = 0, X_1X_2...X_n$ ；

若 $X = -0.X_1X_2...X_n$ ，则 $[X]_{\text{反}} = 1\overline{X_1}\overline{X_2}... \overline{X_n}$



一、数值数据的表示—2.定点数

【例2.16】若机器字长为8，求反码

$$[+35]_{\text{反}} = (00100011)_2$$

$$\begin{aligned} [-35]_{\text{反}} &= (2^8 - 1) + (-35) \\ &= (11111111)_2 - (00100011)_2 \\ &= (11011100)_2 \end{aligned}$$

$$\begin{aligned} 35 &= +00100011 \\ -35 &= -00100011 \end{aligned}$$

绝对值按位取反

$$[+0.8125]_{\text{反}} = (0.1101000)_2$$

$$\begin{aligned} [-0.8125]_{\text{反}} &= (2 - 2^{-7}) + (-0.8125) \\ &= (1.1111111)_2 - (0.1101000)_2 \\ &= (1.0010111)_2 \end{aligned}$$

$$\begin{aligned} 0.8125 &= +0.1101000 \\ -0.8125 &= -0.1101000 \end{aligned}$$



一、数值数据的表示—2.定点数

反码的性质：

- 最高位为符号位，0表示正，1表示负。
- 两种0的表示，使得反码与真值不能一一对应：

$$[+0]_{\text{反}} = 00\dots 0, [-0]_{\text{反}} = 11\dots 1$$

- 表示范围（假设机器字长为 $n+1$ ）：

– 定点小数： $-(1-2^{-n}) \leq X \leq +(1-2^{-n})$

即 $1.00\dots 00$ (n 个0) $\sim$ $0.11\dots 1$ (n 个1)

– 定点整数： $-(2^n-1) \leq X \leq +(2^n-1)$

即 $100\dots 00$ (n 个0) $\sim$ $011\dots 1$ (n 个1)

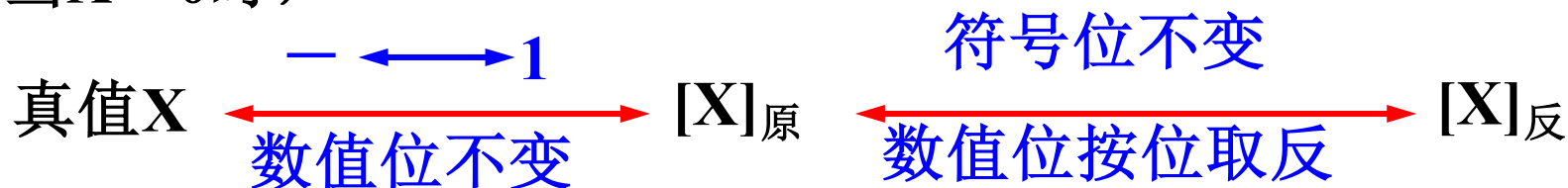
- 负数的反码大于正数的反码。



反码的性质：

- 反码与原码及真值之间的转换：
 - 当 $X \geq 0$ 时，由定义： $[X]_{\text{反}} = [X]_{\text{原}} = X$

- 当 $X < 0$ 时，



- 负数的反码与补码的关系（以小数为例）：

- $[X]_{\text{反}} = 2 - 2^{-n} + X$

- $[X]_{\text{补}} = 2 + X$

可见，**负数的反码末位加1即为其补码（对整数同样成立）。**

	机器数形式	定义
整数	原码	$[X]_{\text{原}} = \begin{cases} 0, X & 0 \leq X < 2^n \\ 2^n - X = 2^n + X & -2^n < X \leq 0 \end{cases}$
	补码	$[X]_{\text{补}} = \begin{cases} 0, X & 0 \leq X < 2^n \\ 2^{n+1} + X & -2^n \leq X < 0 \end{cases} \text{Mod } 2^{n+1}$
	反码	$[X]_{\text{反}} = \begin{cases} 0, X & 0 \leq X < 2^n \\ (2^{n+1} - 1) + X & -2^n < X \leq 0 \end{cases} \text{Mod } (2^{n+1} - 1)$
小数	原码	$[X]_{\text{原}} = \begin{cases} X & 0 \leq X < 1 \\ 1 - X = 1 + X & -1 < X \leq 0 \end{cases}$
	补码	$[X]_{\text{补}} = \begin{cases} X & 0 \leq X < 1 \\ 2 + X & -1 \leq X < 0 \end{cases} \text{Mod } 2$
	反码	$[X]_{\text{反}} = \begin{cases} X & 0 \leq X < 1 \\ (2 - 2^{-n}) + X & -1 < X \leq 0 \end{cases} \text{Mod } (2 - 2^{-n})$

n位机器数	整数可表示范围	小数可表示范围	对应的机器数
原码	$-(2^n - 1) \sim +(2^n - 1)$	$-(1 - 2^{-n}) \sim (1 - 2^{-n})$	111...1~011...1
补码	$-2^n \sim +(2^n - 1)$	$-1 \sim +(1 - 2^{-n})$	100...0~011...1
反码	$-(2^n - 1) \sim +(2^n - 1)$	$-(1 - 2^{-n}) \sim +(1 - 2^{-n})$	100...0~011...1



一、数值数据的表示—2.定点数

下表列出了真值与3种机器数间的对照。表中设字长等于4位(含1位符号位)。

真值X		[X] _原 、[X] _反 、 [X] _补	真值X		[X] _原	[X] _反	[X] _补
十进制	二进制		十进制	二进制			
+0	+ 000	0000	-0	- 000	1000	1111	0000
+1	+ 001	0001	-1	- 001	1001	1110	1111
+2	+ 010	0010	-2	- 010	1010	1101	1110
+3	+ 011	0011	-3	- 011	1011	1100	1101
+4	+ 100	0100	-4	- 100	1100	1011	1100
+5	+ 101	0101	-5	- 101	1101	1010	1011
+6	+ 110	0110	-6	- 110	1110	1001	1010
+7	+ 111	0111	-7	- 111	1111	1000	1001
+8	——	——	-8	- 1000	——	——	1000



一、数值数据的表示—2.定点数

原码、补码和反码这3种机器数的主要区别有以下几点：

- ①对于**正数**它们都等于真值本身，对于**负数**则各有不同的表示。
- ②**最高位都表示符号位**，**补码和反码的符号位可作为数值位的一部分看待，和数值位一起参加运算**；但原码的符号位不允许和数值位同等看待，必须分开进行处理。
- ③对于**真值0**，原码和反码各有两种不同的表示形式，而补码只有唯一的一种表示形式。
- ④**原码、反码表示的正、负数范围相对零来说是对称的**；**补码负数表示范围**较正数表示范围宽，能多表示一个最负的数，其值等于 **-2^n (纯整数)或 -1 (纯小数)**。



一、数值数据的表示—2.定点数

【例2.17】设机器数字长为8位（其中1位为符号位），对于整数，当其分别代表无符号数、原码、补码和反码时，对应的真值范围各为多少？

二进制代码	无符号数 对应的真值	原码对应 的真值	补码对应 的真值	反码对应 的真值
00000000	0	+0	± 0	+0
00000001	1	+1	+1	+1
00000010	2	+2	+2	+2
01111110	126	+126	+126	+126
01111111	127	+127	+127	+127
10000000	128	-0	-128	-127
10000001	129	-1	-127	-126
10000010	130	-2	-126	-125
:	:	:	:	:
11111101	253	-125	-3	-2
11111110	254	-126	-2	-1
11111111	255	-127	-1	-0



一、数值数据的表示—2.定点数

【例2.17*】设机器数字长为8位（其中1位为符号位），对于小数，当其分别代表无符号数、原码、补码和反码时，对应的真值范围各为多少？

二进制代码	原码对应的真值	补码对应的真值
00000000	0	± 0
00000001	2^{-7}	2^{-7}
00000010	2^{-6}	2^{-6}
:		
01111111	$1-2^{-7}$	$1-2^{-7}$
10000000	-0	-1
10000001	-2^{-7}	$-(1-2^{-7})$
10000010	-2^{-6}	$-(1-2^{-6})$
:	:	:
11111111	$-(1-2^{-7})$	-2^{-7}



一、数值数据的表示—2.定点数

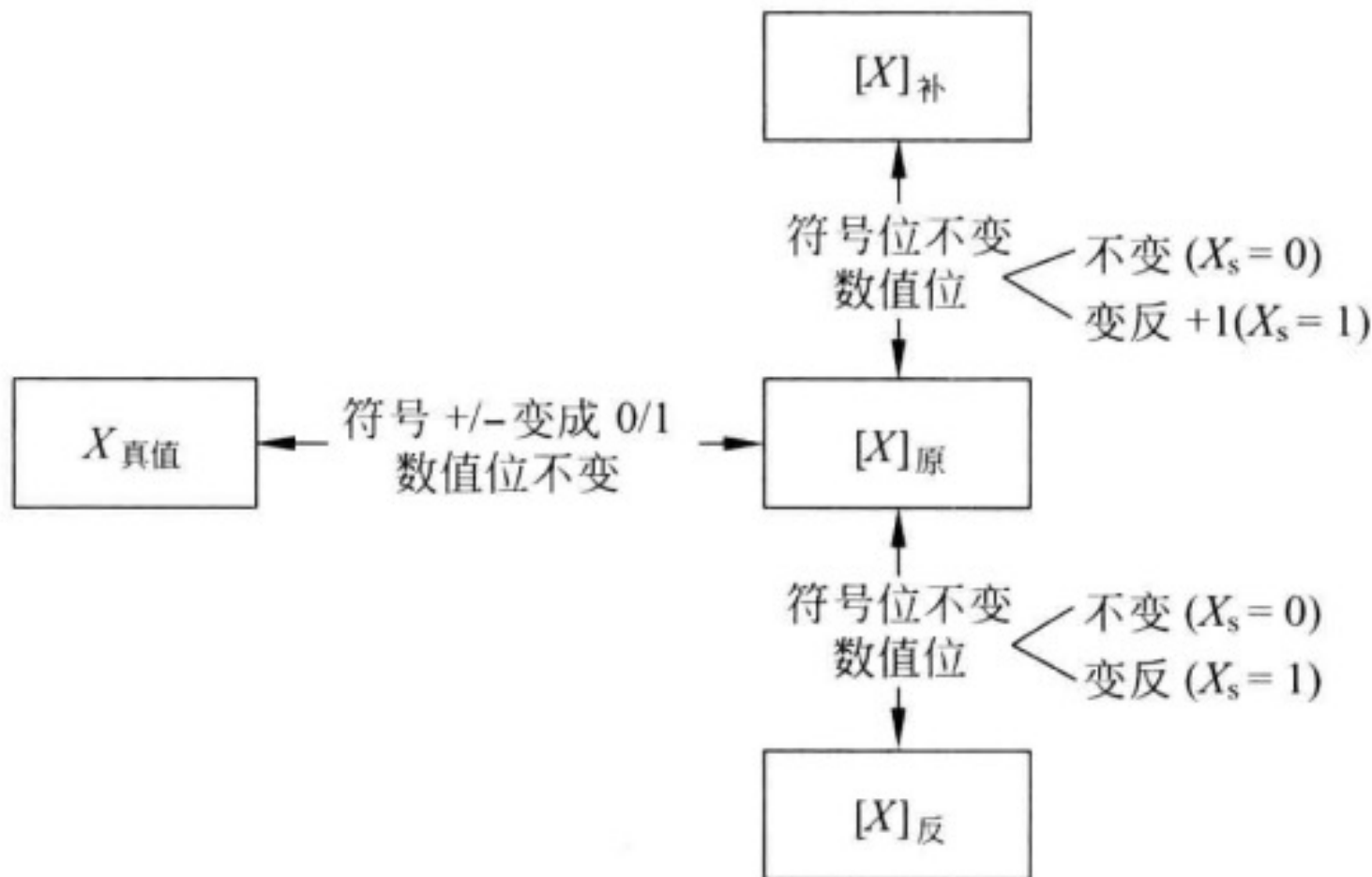
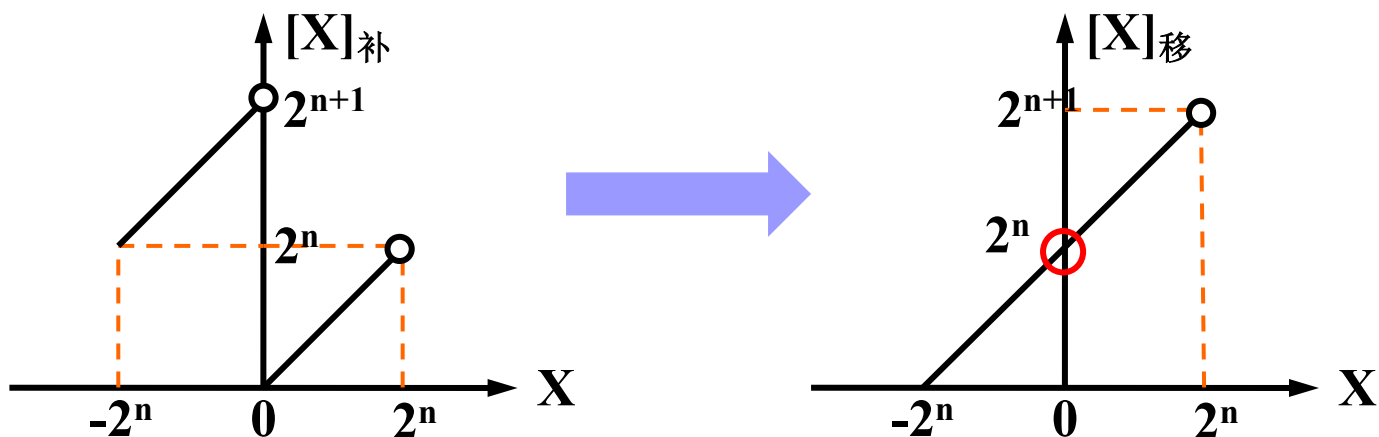


图 2-1 3 种不同机器数及真值间的转换关系



2.4 移码表示

- 以整数为例，在数值上，负数的补码比正数的大，而给每个数都加一个偏移量 2^n 后，补码与真值的关系发生了如下变化：



- 即把负端的数据全部移到了正端，因此被称为“移码”，又叫“增码”。



一、数值数据的表示—2.定点数

- 计算机中常用移码来表示浮点数的阶码 → 即指数。阶码为整数，故我们只讨论定点整数的移码表示。

- 机器字长为 $n+1$ 位，则移码定义：

$$[X]_{\text{移}} = 2^n + X, \quad -2^n \leq X < 2^n$$

- 由于 $[X]_{\text{补}} = 2^{n+1} + X \pmod{2^{n+1}}$

$$[X]_{\text{移}} = 2^n + [X]_{\text{补}} \pmod{2^{n+1}}, \quad -2^n \leq X \leq 2^n - 1$$

结论：求移码的表示形式只需将补码的符号位取反即可。

- 若 $X = +X_1X_2...X_n$ ，则 $[X]_{\text{移}} = 1X_1X_2...X_n$ ；

若 $X = -X_1X_2...X_n$ ，则 $[X]_{\text{移}} = 0\overline{X_1}\overline{X_2}...\overline{X_n} + 1$



一、数值数据的表示—2.定点数

【例2.16】假设机器字长为8，已知

$$X_1 = +101011, X_2 = -110010,$$

求其原码、补码和移码。

【解】

$$[X_1]_{\text{原}} = 00101011$$

$$[X_2]_{\text{原}} = 10110010$$

$$[X_1]_{\text{补}} = 00101011$$

$$[X_2]_{\text{补}} = 11001110$$

$$[X_1]_{\text{移}} = 10101011$$

$$[X_2]_{\text{移}} = 01001110$$

补码与移码只差一个符号位



移码的性质：

- 移码与其他三种机器数有本质的区别。
 - 符号位：“0”表示负，“1”表示正。
 - 只表示整数，不表示小数。表示范围： $n+1$ 位机器字长，
 $-2^n \leq X \leq 2^n - 1$
 - 建议将移码视为无符号数，不要将其最高位当做符号位。
- “0”的移码表示是唯一的。

$$[+0]_{\text{移}} = [-0]_{\text{移}} = 2^n + 0 = 1 \underbrace{00 \cdots 0}_{n \uparrow 0}$$



一、数值数据的表示—2.定点数

定点整数 X 与 $[X]_{\text{原}}$ 、 $[X]_{\text{补}}$ 、 $[X]_{\text{移}}$ 的对应关系（机器字长为8）

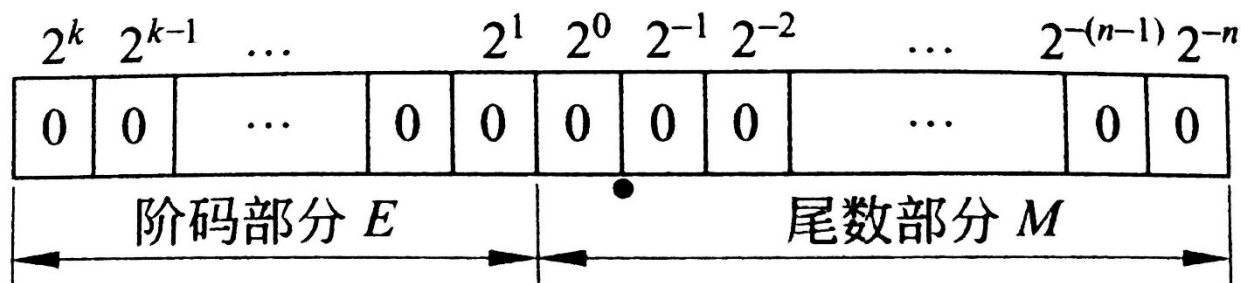
真值 X		X 的原码 表示 $[X]_{\text{原}}$	X 的补码 表示 $[X]_{\text{补}}$	X 的移码表示 $[X]_{\text{移}}$	
十进制 表示	二进制表示			二进制表示	十进制 表示
+127	+0111 1111	0111 1111	0111 1111	1111 1111	255
+126	+0111 1110	0111 1110	0111 1110	1111 1110	254
.....					
+2	+0000 0010	00000010	0000 0010	1000 0010	130
+1	+0000 0001	00000001	0000 0001	1000 0001	129
0	0000 0000	0000 0000 1000 0000	0000 0000	1000 0000	128
-1	-0000 0001	1000 0001	1111 1111	0111 1111	127
-2	-0000 0010	1000 0010	1111 1110	0111 1110	126



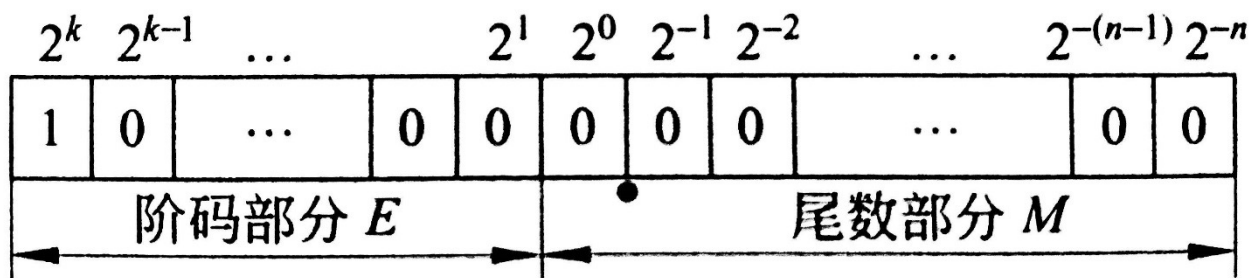
一、数值数据的表示—2.定点数

浮点数的阶码采用移码与采用补码相比，具有两点优势

- ① 便于比较浮点数的大小。
- ② 可以简化浮点运算中的对阶操作。



(a) 阶码用移码表示的机器零表示形式



(b) 阶码用补码表示的机器零表示形式

(1) 原码: 假设 $[X]_{\text{原}} = X_0, X_1 X_2 \dots X_n$

若 $X_0 = 0$, 则 $X = + X_1 X_2 \dots X_n$

若 $X_0 = 1$, 则 $X = - X_1 X_2 \dots X_n$

(2) 补码: 假设 $[X]_{\text{补}} = X_0, X_1 X_2 \dots X_n$

若 $X_0 = 0$, 则 $X = + X_1 X_2 \dots X_n$

若 $X_0 = 1$, 则 $X = - (\overline{X_1} \overline{X_2} \dots \overline{X_n} + 1)$

(3) 反码: 假设 $[X]_{\text{反}} = X_0, X_1 X_2 \dots X_n$

若 $X_0 = 0$, 则 $X = + X_1 X_2 \dots X_n$

若 $X_0 = 1$, 则 $X = - \overline{X_1} \overline{X_2} \dots \overline{X_n}$

(4) 移码: 假设 $[X]_{\text{移}} = X_0, X_1 X_2 \dots X_n$

若 $X_0 = 1$, 则 $X = + X_1 X_2 \dots X_n$

若 $X_0 = 0$, 则 $X = - (\overline{X_1} \overline{X_2} \dots \overline{X_n} + 1)$



一、数值数据的表示

1. 数据格式
2. 定点数表示
3. 浮点数表示
4. 计算机中的数据类型

二、非数值数据的表示

1. 字符码
2. 汉字编码
3. Unicode编码

三、数据信息的校验

1. 码距与数据校验
2. 奇偶校验
3. 海明校验
4. 循环冗余校验



一、数值数据的表示—3.浮点数

3.1 浮点表示

- 对于任意一个二进制数 X ,可以表示成如下形式:

$$X=(-1)^s \times M \times R^E$$

- S 取值为0或1, 用来决定数 X 的符号
- M 是一个二进制定点小数, 称为数 X 的尾数(mantissa)
- E 是一个二进制定点整数, 称为数 X 的阶或指数(exponent)
- R 是基数(radix, base), 可以取值为2、4、16等。

在基数 R 一定的情况下

- 尾数 M 的位数反映数 X 的有效位数, 它决定了数的表示精度, 有效位数越多, 表示精度就越高;
- 指数 E 的位数决定数 X 的表示范围, 其值确定了小数点的位置。

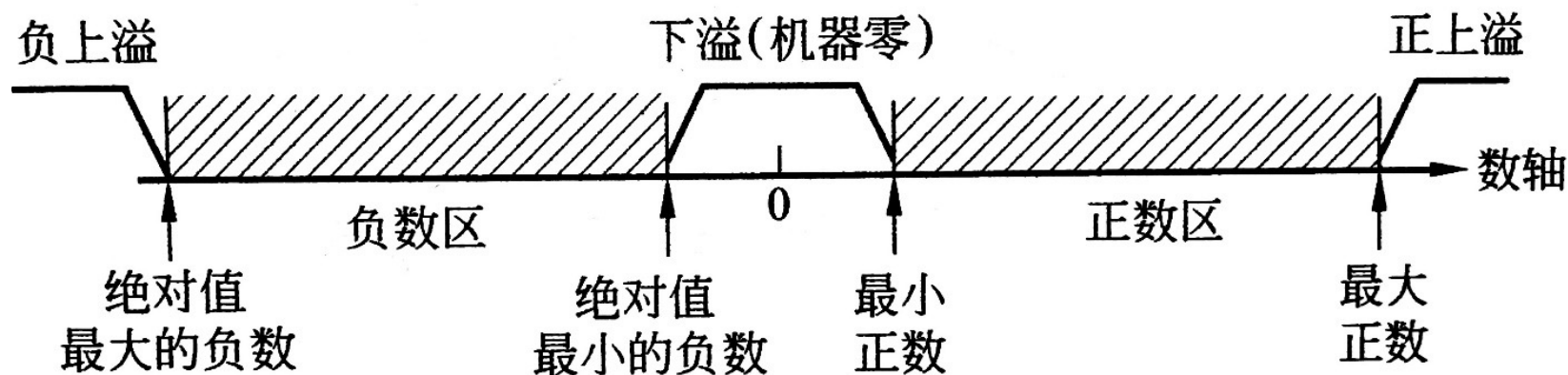


一、数值数据的表示—3.浮点数

浮点数表示的范围

浮点数的表示范围通常由四个点界定：

- 最小（负）数/绝对值最大的负数
- 最大负数/绝对值最小的负数
- 最小正数——分辨率
- 最大（正）数。



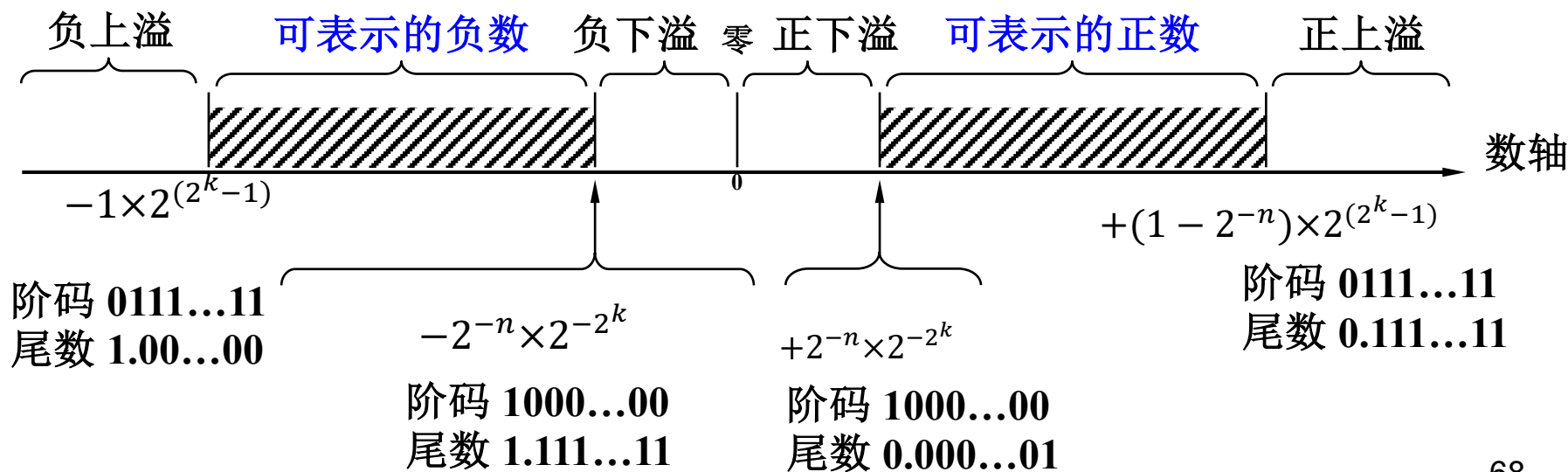


一、数值数据的表示—3.浮点数

浮点数表示范围:

假设浮点数的尾数部分 ($n+1$ 位) 和阶码部分 ($k+1$ 位) 均用补码表示。根据定点小数和整数的基础, 浮点数的表示范围为:

阶码和非规格化尾数 (补码编码) 的数值范围			
阶码最小值	-2^k	阶码最大值	2^k-1
尾数最小负值	-1	尾数最大负值	-2^{-n}
尾数最小正值	$+2^{-n}$	尾数最大正值	$+(1-2^{-n})$





一、数值数据的表示—3.浮点数

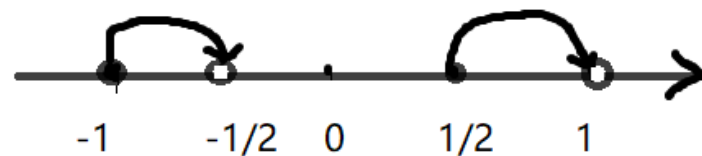
3.2 规格化浮点数

- 尾数的最高位必须是一个有效值。真值的绝对值 $\geq 1/R$ 。
 - 注意：假设尾数的基数为2，有效值不是指机器数（原码或补码）的最高数位为1，而是指真值的最高数位必须是1。
 - 正数：0.1XX...X（无论原码或补码） $\frac{1}{2} \leq M < 1$
 - 负数：1.1XX...X（原码），对应的真值为 $-1 < M \leq -\frac{1}{2}$ ；
1.0XX...X（补码），对应的真值为 $-1 \leq M < -\frac{1}{2}$
 - 尾数用补码表示，判断一个数是否规格化，可根据尾数的符号位和最高数值位是否相异决定。若两者相异，即为规格化数。



一、数值数据的表示—3.浮点数

- 当尾数用补码表示时



– 若尾数 $M \geq 0$ ，由于 $[1/2]_{\text{补}} = 0.1000\dots 0$ ，

∴ 尾数应具有格式： $M = 0.1\text{xxx}\dots\text{x}$

∴ 当 $M \geq 0$ 时， $[1/2]_{\text{补}} \leq [M]_{\text{补}} < [1]_{\text{补}}$

– 若尾数 $M < 0$ ，由于 $[-1/2]_{\text{补}} = 1.1000\dots 0$ ， $[-1]_{\text{补}} = 1.000\dots 0$

为了使计算机判断方便，一般不把 $[-1/2]_{\text{补}}$ 列为规格化的数，而把 $[-1]_{\text{补}}$ 列为规格化的数

∴ 尾数应具有格式： $M = 1.0\text{xxx}\dots\text{x}$

∴ 当 $M < 0$ ， $[-1]_{\text{补}} \leq [M]_{\text{补}} < [1/2]_{\text{补}}$

异或逻辑

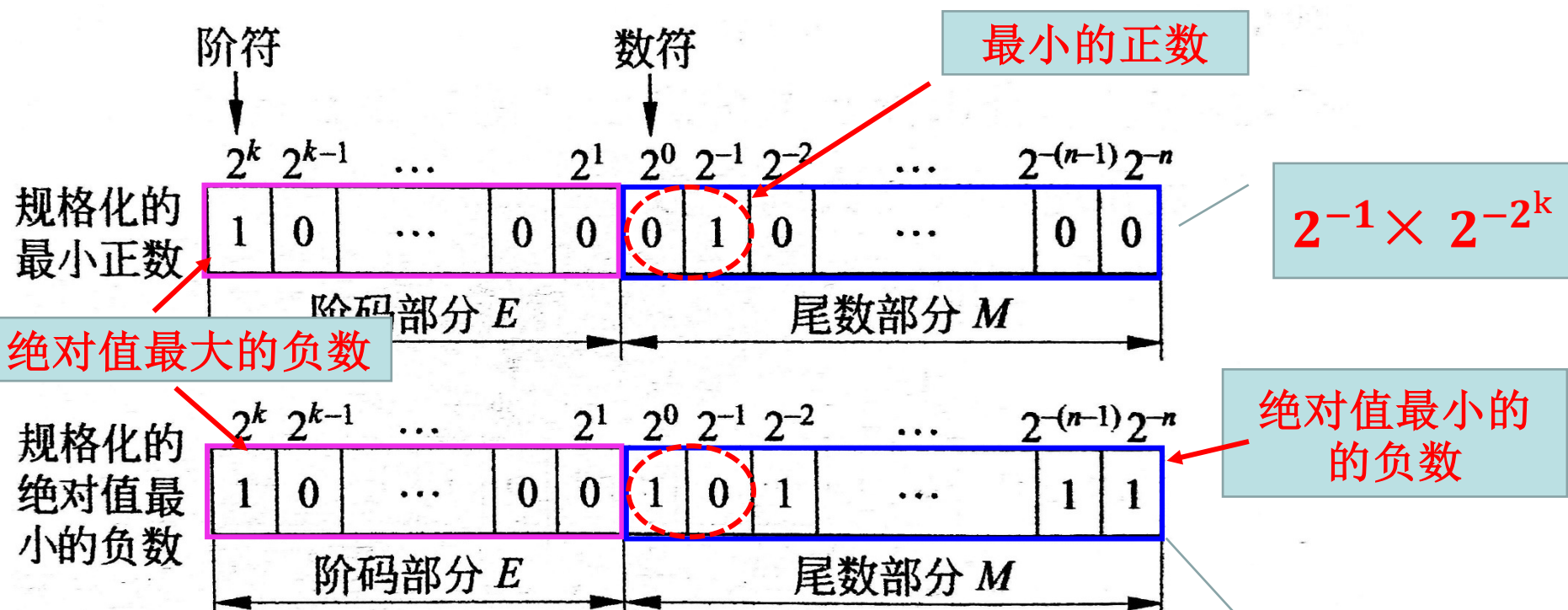
1.1000...0	(-1/2)
1.0111...1	
1.0111...0	
.....	
1.0000...1	
1.0000...0	(-1)





一、数值数据的表示—3.浮点数

设浮点数的尾数和阶码均用补码表示，规格化的最小正数和规格化的绝对值最小负数的表示形式如图所示。



规格化的最小正数和规格化的绝对值最小的负数的表示形式

$$-(2^{-1} + 2^{-n}) \times 2^{2^k}$$



一、数值数据的表示—3.浮点数

- 对于非规格化浮点数，可以通过**修改阶码**和**左右移尾数**的方法来使其变为规格化浮点数，这个过程叫做**规格化**；
 - 若**尾数进行右移**实现的规格化，则称为**右规**；**阶码加1**
 - 若**尾数进行左移**实现的规格化，则称为**左规**。**阶码减1**
- 采用两位符号位（变形补码），当结果尾数的两个符号位不一致时，表明**结果溢出**。**需要右规1次**。当**结果不溢出**，但**高位和数值位同值**，表明不满足规格化规则，此时**重复地左规**，直到满足规格化规则。

- | | | |
|--------------|---|---------|
| ① 00.1×××××× | } | 规格化的浮点数 |
| ② 11.0×××××× | | |
| ③ 00.0×××××× | } | 需要左规若干次 |
| ④ 11.1×××××× | | |
| ⑤ 01.×××××× | } | 需要右规1次 |
| ⑥ 10.×××××× | | |



一、数值数据的表示—3.浮点数

※ 【例2.18】 将十进制数 $x=+13/128$ 写成二进制定点数和浮点数（尾数数值部分取7位，阶码数值部分取7位，阶码和数码各取1位，**阶码采用移码，尾数用补码表示**），并分别写出定点数和浮点数的编码。

解： $x=+13/128_{10} = 1101_2 \div 2^7 = 0.0001101_2 = 0.1101 \times 2^{-11}$

定点数编码为： $[x]_{\text{原}} = [x]_{\text{补}} = [x]_{\text{反}} = 0.0001101_2$

$-3_{10} = -00000011_2$

$[-3]_{\text{补}} = 11111101_2$

$[-3]_{\text{移}} = 01111101_2$

规格化浮点数编码为

数符	阶符	阶码	尾数
0	0	1111101	1101000



3.3 浮点数表示的范围与精度

浮点数的表示范围通常由四个点界定：

- 最小（负）数/绝对值最大的负数
- 最大负数/绝对值最小的负数
- 最小正数——分辨率
- 最大（正）数。

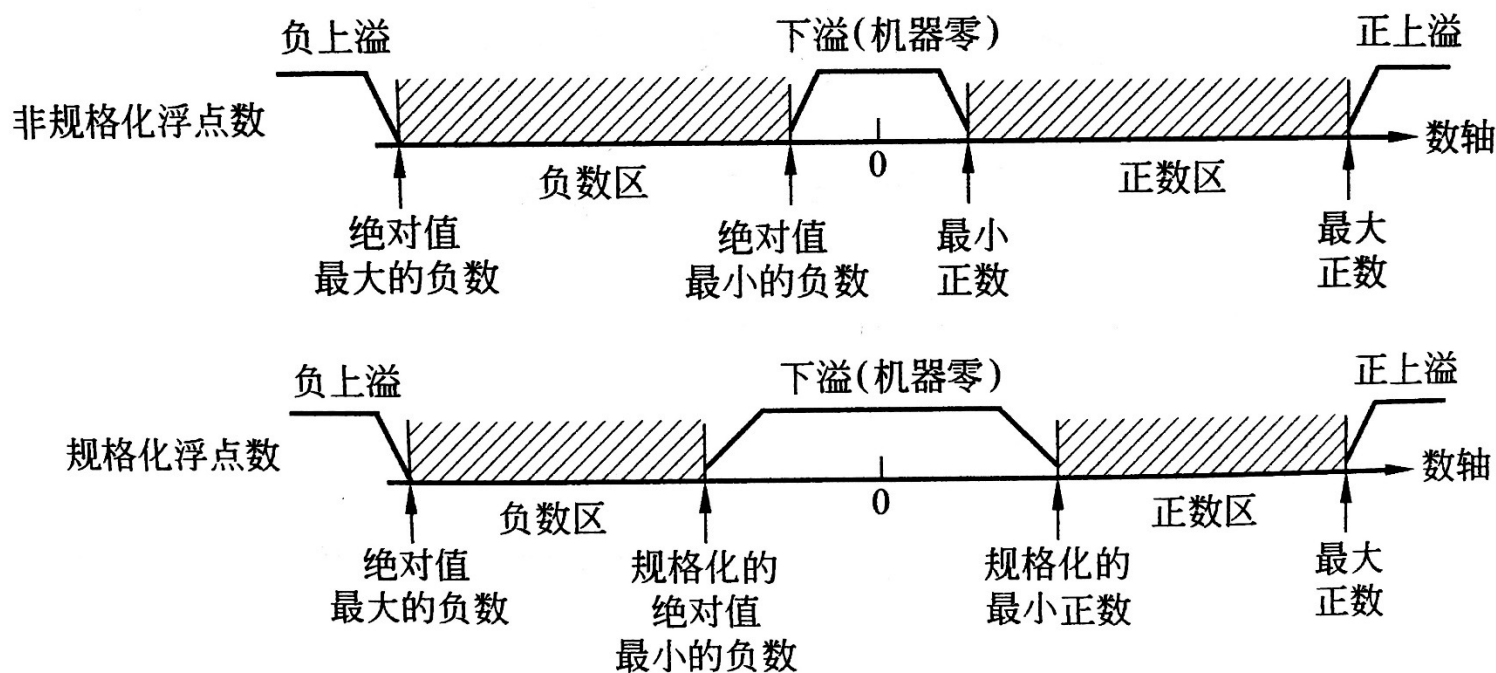
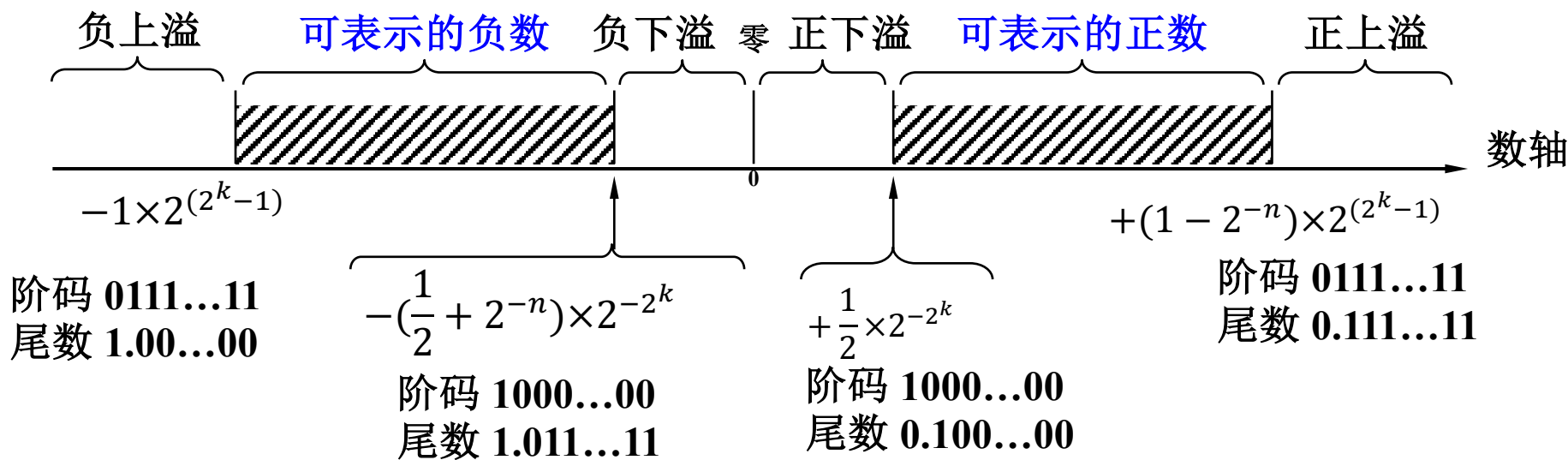


图 2-14 浮点数的表示范围

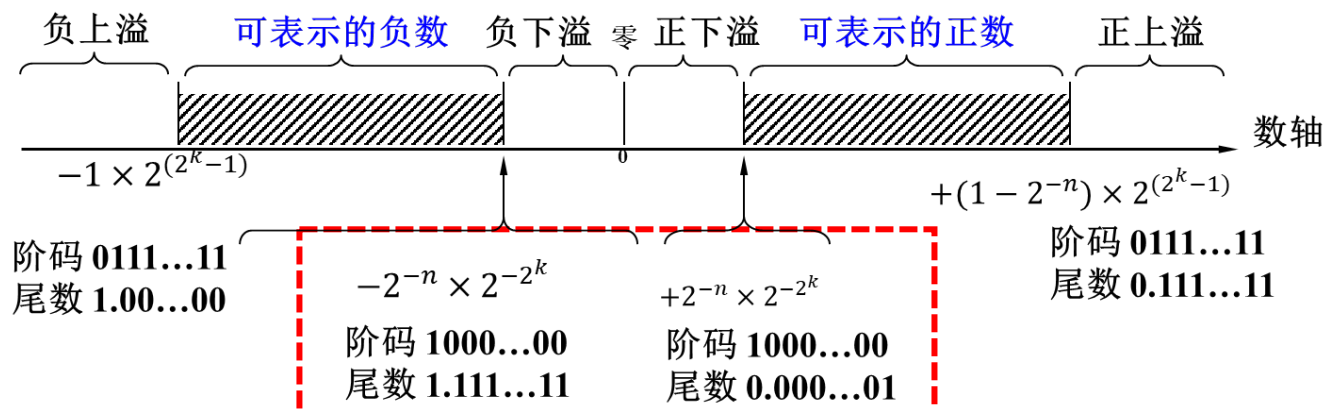


一、数值数据的表示—3.浮点数

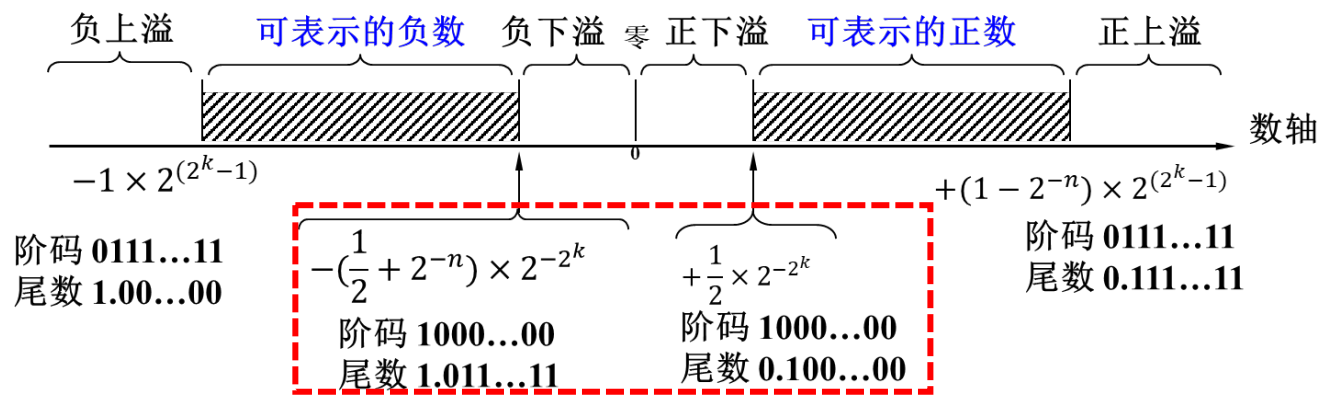
阶码（补码）和规格化尾数（补码编码）的数值范围			
阶码最小值	-2^k	阶码最大值	2^k-1
尾数最小负值	-1	尾数最大负值	$-(1/2+2^{-n})$
尾数最小正值	$+1/2$	尾数最大正值	$+(1-2^{-n})$



阶码和 非规格化尾数 （补码编码）的数值范围			
阶码最小值	-2^k	阶码最大值	2^k-1
尾数最小负值	-1	尾数最大负值	-2^{-n}
尾数最小正值	$+2^{-n}$	尾数最大正值	$+(1-2^{-n})$



阶码和 规格化尾数 （补码编码）的数值范围			
阶码最小值	-2^k	阶码最大值	2^k-1
尾数最小负值	-1	尾数最大负值	$-(1/2+2^{-n})$
尾数最小正值	$+1/2$	尾数最大正值	$+(1-2^{-n})$



- 设浮点数字长16位，基底为2。其中**阶码6位**(含1位阶符)，用移码表示；**尾数10位**(含1位数符)，用补码表示。
- 阶码范围：-32 (-2^5) ~ 31 (2^5-1)
- 尾数范围：规格化：-1 ~ - ($2^{-1}+2^{-9}$)， $2^{-1} \sim (1-2^{-9})$
非规格化：-1 ~ - 2^{-9} ， $2^{-9} \sim (1-2^{-9})$

(1) 规格化表示范围

	真值	浮点数表示
最小数	$-1 \times 2^{31} = -2^{31}$	1, 11111 1.000000000
最大负数	$-(2^{-1} + 2^{-9}) \times 2^{32}$	0, 00000 1.011111111
最小正数	$2^{-1} \times 2^{-32} = 2^{-33}$	0, 00000 0.100000000
最大数	$(1-2^{-9}) \times 2^{31}$	1, 11111 0.111111111

(2) 非规格化表示范围

	真值	浮点数表示
最小数	$-1 \times 2^{31} = -2^{31}$	1, 11111 1.000000000
最大负数	$-2^{-9} \times 2^{-32} = -2^{-41}$	0, 00000 1.111111111
最小正数	$2^{-9} \times 2^{-32} = 2^{-41}$	0, 00000 0.000000001
最大数	$(1-2^{-9}) \times 2^{31}$	1, 11111 0.111111111



【例2.19】比较字长为32位的定点整数和浮点数（规格化，阶码部分8位）的表示范围和精度，均采用补码表示。

解：若定点整数32位，补码表示，则表示范围为 $-2^{31} \sim 2^{31}-1$ ，分辨率为1。

若浮点数32位，其中阶码部分8位，含1位阶符，补码表示，以2为底；尾数部分24位，含1位数符，补码表示，规格化，则表示范围为 $-2^{127} \sim 2^{127} \times (1-2^{-23})$ ，最高分辨率为 2^{-129} 。

说明：用n位二进制数表示的浮点数，能表示不同值的最大数目仍是 2^n ，只是把这些数沿数轴正负两个方向在更大范围内散布开。浮点表示的数不再像定点数那样沿数轴等距分布，而是越靠近原点，数越密集；越远离原点，数越稀疏。



一、数值数据的表示—3.浮点数

3.4 IEEE 754标准

- 浮点数的表示标准：IEEE 754，1985年由IEEE制定。
- 目的：便于程序从一类处理器移植到另一类处理器上。

IEEE标准浮点数阶码的偏置值是同样位数移码的偏置值减1的原因：IEEE 754规格化浮点数隐含了最高数位，即相当于尾数扩大了一倍（左移1位），为保持该浮点数的值不变，阶码应当相应地减1。

- 规定
 - 基值隐含为2。
 - 尾数用原码表示，小数点前隐含一个“1”，取值范围[1,2)。
 - 阶码用移码表示，移码的偏移量有专门约定：n位移码的偏置值为 $2^{n-1}-1$ 。（注意和移码定义区分）。
 - 指数（阶码）的最大值、最小值作为特殊标记预留，用来标记某些异常事件或机器零。
 - 单精度(float)、双精度(double)：单精度扩展、双精度扩展。

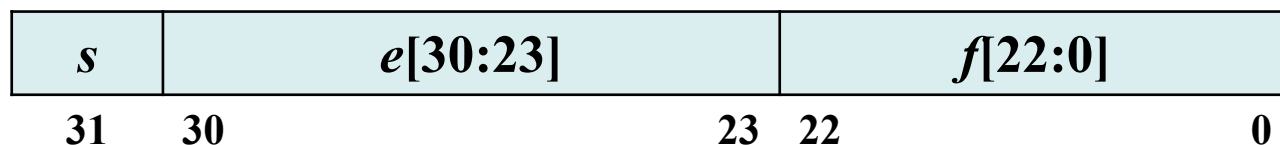


一、数值数据的表示—3.浮点数

- IEEE 754标准浮点数（单精度格式）的真值：

$$(-1)^s \times (1.f) \times 2^{e-127}$$

- 单精度格式：三个字段组成：23位的尾数 f （实际上是24位），8位移码表示的指数 e （偏移值为 $2^7-1=127$ ），以及1位符号 s 。



- 最高位是符号位——判断正负与定点数完全一致。
- 尾数用原码表示，隐含了数据最高位的1——增加了尾数的位数，提高了精度
- 当阶码部分和尾数部分均为全0时，结果为0——判0方便。尾数用原码表示，所以0有两种表示：+0，-0。0的符号取决于IEEE 754格式的最高位。
- 无法表示0和及其靠近0的数：32位 IEEE 754取值范围 $\pm [1,2) \times 2^{[-126, 127]}$ ⁸⁰



一、数值数据的表示—3.浮点数

IEEE 754 标准的单精度和双精度格式的特征参数		
参数	单精度浮点数	双精度浮点数
字宽（位数）	32	64
阶码宽度（位数）	8	11
阶码偏置常数	127	1023
最大指数	127	1023
最小指数	-126	-1022
尾数宽度	23	52
阶码个数	254	2046
尾数个数	2^{23}	2^{52}
值的个数	1.98×2^{31}	1.99×2^{63}
数的量级范围	$10^{-38} \sim 10^{+38}$	$10^{-308} \sim 10^{+308}$



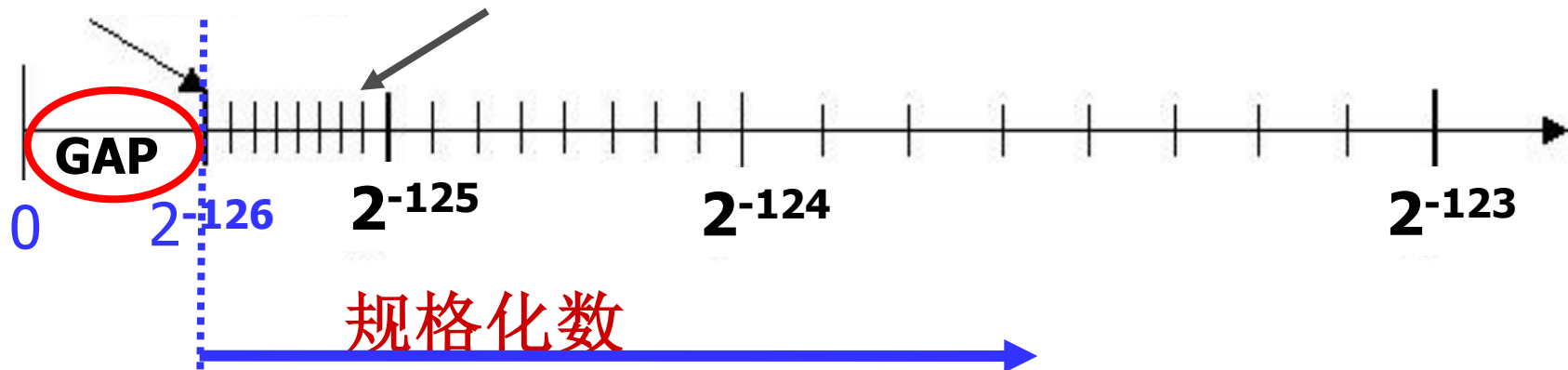
IEEE 754单精度浮点数的极限情况定义见表

名称	符号	阶码域	尾数域	阶码(含偏移)	阶(指数)	数值(十进制)
正零	0	0000 0000	000 0000 0000 0000 0000 0000	0	-127	0
负零	1	0000 0000	000 0000 0000 0000 0000 0000	0	-127	-0
1	0	0111 1111	000 0000 0000 0000 0000 0000	127	0	1
-1	1	0111 1111	000 0000 0000 0000 0000 0000	127	0	-1
最小规格化数	0/1	0000 0001	000 0000 0000 0000 0000 0000	1	-126	$\pm 1.2 \times 10^{-38}$
最大规格化数	0/1	1111 1110	111 1111 1111 1111 1111 1111	254	127	$\pm 3.4 \times 10^{-38}$
正无穷	0	1111 1111	000 0000 0000 0000 0000 0000	255	128	$+\infty$
负无穷	1	1111 1111	000 0000 0000 0000 0000 0000	255	128	$-\infty$
非数 (NaN)	0/1	1111 1111	非全0	255	128	NaN
非规格化数	0/1	0000 0000	高位有1个或几个连续0，但不全为0，隐含位为0	0	-126	$(-1)^s \times 0.f \times 2^{-126}$

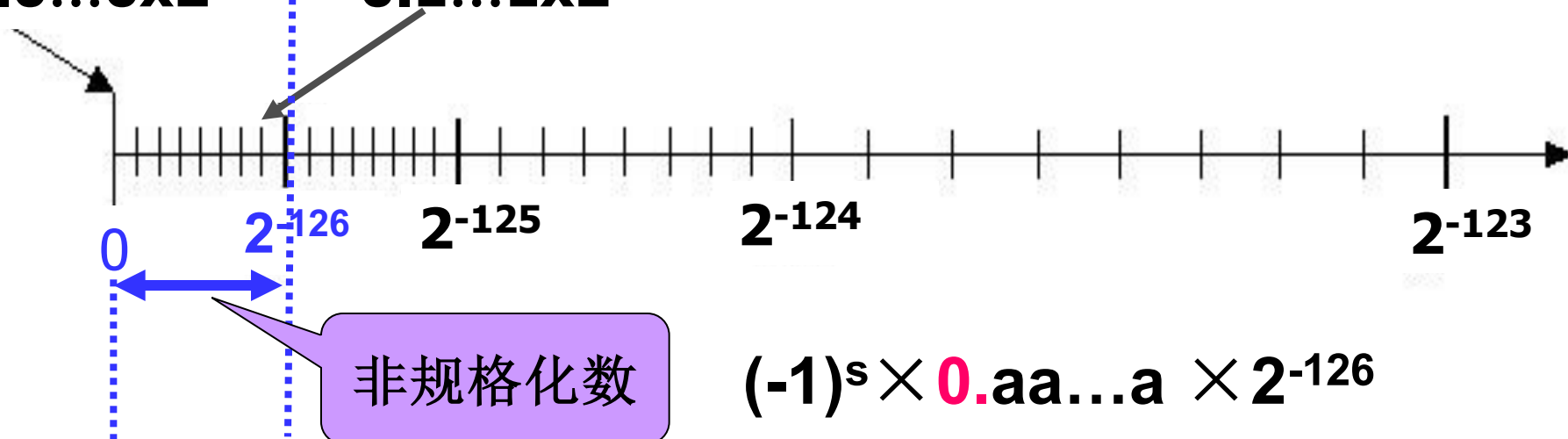


一、数值数据的表示—3.浮点数

$1.0...0 \times 2^{-126} \sim 1.1...1 \times 2^{-126}$



$0.0...0 \times 2^{-126} \sim 0.1...1 \times 2^{-126}$



$$(-1)^s \times 0.a \dots a \times 2^{-126}$$



【例2.21】若浮点数x的IEEE754单精度存储格式为 $(42150000)_{16}$ ，求浮点数x的十进制真值。

解：将16进制数展开后，可得二进制数格式如下：



$$\text{指数} = e - (127)_{10}$$

$$= (10000100)_2 - (127)_{10} = (132)_{10} - (127)_{10} = (5)_{10}$$

$$\text{包含隐含位1的尾数} = 1.f$$

$$= (1.001\ 0101\ 0000\ 0000\ 0000\ 0000)_2 = (1.001\ 0101)_2$$

$$\text{因此, } x = (-1)^s \times (1.f) \times 2^{e-127}$$

$$= +(1.001010)_2 \times 2^5 = (100101.01)_2 = (37.25)_{10}$$



一、数值数据的表示—3.浮点数

【例2.22】将数 $(22.78125)_{10}$ 转换成IEEE 754单精度浮点数的二进制存储格式。

解： $(22.78125)_{10} = (10110.11001)_2 = (1.011011001)_2 \times 2^4$

符号位 $s = 0$

阶码 $e = \text{指数} + (127)_{10} = (4 + 127)_{10} = (10000011)_2$

尾数 $f = (011011001)_2$

∴ 32位单精度浮点数的二进制存储格式为：

$(0100\ 0001\ 1011\ 0110\ 0100\ 0000\ 0000\ 0000)_2 = (41B64000)_{16}$



一、数值数据的表示

1. 数据格式
2. 定点数表示
3. 浮点数表示
4. 计算机中的数据类型

二、非数值数据的表示

1. 字符码
2. 汉字编码
3. Unicode编码

三、数据信息的校验

1. 码距与数据校验
2. 奇偶校验
3. 海明校验
4. 循环冗余校验



一、数值数据的表示

4. 计算机中的数据类型

1. 汇编语言中的数据类型

- 汇编语言中的变量就是寄存器和存储器中的操作数。操作数到底是定点数还是浮点数，是有符号数还是无符号数完全取决于指令操作符。
- x86指令集中常见的ADD、SUB、MOV等指令；如果是浮点指令，如FADD、FSUB、FMUL、FDIV分别表示浮点加、减、乘、除，则指令处理的操作数是浮点数据。
- 下表给出了汇编语言中不同指令集的数据运算类型：

ISA	无符号运算	有符号运算	浮点运算
x86	ADD/SUB 加减		FADD/FSUB 加减
	MUL/DIV 乘除	IMUL/IDIV 乘除	FMUL/FDIV 乘除
MIPS32	ADDU/SUBU 加减	ADD/SUB 加减	ADD.S ADD.D/SUB.S SUB.D 加减
	MULTU/DIVU 乘除	MULT/DIV 乘除	MUL.S MUL.D/DIV.S DIV.D 乘除
RISC-V32	ADD/SUB 加减		FADD.S FADD.D/FSUB.S FSUB.D 加减
	MULHU/DIVU 乘除	MULH/DIV 乘除	FMUL.S FMUL.D/FDIV.S FDIV.D 乘除



一、数值数据的表示

2. 高级语言中的数据类型

C声明		字节数	
有符号	无符号	32位	64位
[signed] char	unsigned char	1	1
short	unsigned short	2	2
int	unsigned	4	4
long	unsigned long	4	8
char *		4	8
float		4	8
double		8	8

- 有符号整数采用补码表示和存储。
- 无符号数据多一个符号位，可用于表示数据。
- float和double分别对应IEEE 754中的单精度和双精度浮点数。
- 不同数据类型的运算会在编译器的翻译下变成不同类型的汇编指令，如有符号乘法和无符号乘法对应不同的汇编指令，整数加法和浮点加法的汇编指令也是不一样的。



一、数值数据的表示

(1) C语言整型数据表示范围

- 无论是无符号数还是有符号数，实际上C语言程序并不检测数据在加、减、乘等运算中产生的溢出现象。程序员应尽量避免出现这种情况，对溢出应通过程序进行判断。

表 2.12 整型变量的取值范围

值		char (8 位)	short (16 位)	int (32 位)	long (64 位)
无符号 最大值	机器码	0xFF	0x FFFF	0x FFFF FFFF	0x FFFF FFFF FFFF FFFF
	真值	255	65,535	4,294,967,295	18,446,744,073,709,551,615
有符号 最大值	机器码	0x7F	0x 7FFF	0x 7FFF FFFF	0x 7FFF FFFF FFFF FFFF
	真值	127	32,767	2,147,483,647	9,223,372,036,854,775,807
有符号 最小值	机器码	0x80	0x 8000	0x 8000 0000	0x 8000 0000 0000 0000
	真值	-128	-32,768	-2,147,483,648	-9,223,372,036,854,775,808
-1	机器码	0xFF	0x FFFF	0x FFFF FFFF	0x FFFF FFFF FFFF FFFF
0	机器码	0x00	0x 0000	0x 0000 0000	0x 0000 0000 0000 0000



一、数值数据的表示

(2) C语言整型数据类型转换

不同类型的数据可以互相进行强制类型转换，基本转换原则是**尽量保持数的真值不变**。

- **相同位宽的整型数据进行强制转换时，机器码保持不变；**

此时只是有符号类型和无符号类型之间的相互转换。以8位整型数为例，8位无符号整型数据的表示范围是 $[0, 255]$ ，而8位有符号整数的表示范围是 $[-128, 127]$ ，如果要转换的数据在二者的交集中，也就是在 $[0, 127]$ 中，则转换后的数据与原值相同，否则就会出现比较奇怪的现象。

- **小字长转大字长时，如果原数据是无符号类型，则进行零扩展；否则进行符号扩展，扩展数据的高位部分利用原数据符号位填充。**
- **大字长转小字长时，此时大概率会损失表示范围，转换时要格外小心，通常编译器会直接将机器码截短处理。**



一、数值数据的表示

1. 数据格式
2. 定点数表示
3. 浮点数表示
4. 计算机中的数据类型

二、非数值数据的表示

1. 字符码
2. 汉字编码
3. Unicode编码

三、数据信息的校验

1. 码距与数据校验
2. 奇偶校验
3. 海明校验
4. 循环冗余校验



二、非数值数据的表示

1 字符码

- ASCII字符编码

美国国家信息交换标准代码（American Standard Code for Information Interchange，简称ASCII码）是广泛使用的一种字符编码方案，1967年，成为官方标准，即ASCII码。

ASCII码选用了128个常用字符及控制码，用7位二进制位编码，通常用一个字节来表示。其中，最高位预留作为奇偶校验位，用于数据传输过程中的错误检测，在机器中通常使其为0。

$b_3b_2b_1b_0$	$b_6b_5b_4$							
	000	001	010	011	100	101	110	111
0000	NUL	DLE	(SPACE)	0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL



二、非数值数据的表示

- 0100000 (20H) 开始是空格等可打印字符, 0~9这10个数字是从0110000 (30H) 开始的一个连续区域, 大写英文字母是从1000001 (41H) 开始的一个连续区域, 小写英文字母是从1100001 (61H) 开始的一个连续区域。
- 在数码转换时, 可以利用上述连续编码的特性, 从一个ASCII的编码求出另一个ASCII的编码。例如。将5转换成ASCII码时, 只需将0的ASCII字符30H加上5即可。同理, 计算英文字符的ASCII编码也只需要记住“A”和“a”的ASCII编码即可。



2. 汉字编码

① 汉字国际码

- 汉字数量众多，为此汉字编码采用了**双字节编码**，为与ASCII编码兼容并区分，汉字编码**双字节的最高位MSB都为1**，也就是**实际使用了14位来表示汉字**。这就是**1980年颁布的国家标准GB2312**，也称**国标码**。
- **GB2312编码理论上能表示 $2^{14}=16384$ 个编码**，而实际上仅包含了**7445个字符**，其中**6763个常用汉字**、**682个全角非汉字字符**。



二、非数值数据的表示

② 区位码

为了检索方便，该标准采用 $94 \times 94 = 8836$ 的二维矩阵对字符集中的所有汉字字符进行了编码，矩阵的每一行称为“区”，每一列称为“位”，区号和位号都从1开始编码，采用十进制表示，所有字符都在矩阵中有唯一的位置，这个位置可以用区号和位号组合表示，称为汉字的区位码，区位码“1818”是“膊”字，如图所示。

区位码	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
16区	啊	阿	埃	挨	哎	唉	哀	皑	癌	蔼	矮	艾	碍	爱	隘	鞍	氨	安	俺	按
17区	薄	雹	保	堡	饱	宝	抱	报	暴	豹	鲍	爆	杯	碑	悲	卑	北	辈	背	贝
18区	病	并	玻	菠	播	拨	钵	波	博	勃	搏	铂	箔	伯	帛	舶	脖	膊	渤	泊
19区	场	尝	常	长	偿	肠	厂	敞	畅	唱	倡	超	抄	钞	朝	嘲	潮	巢	吵	炒
20区	础	储	矗	搐	触	处	揣	川	穿	椽	传	船	喘	串	疮	窗	幢	床	闯	创



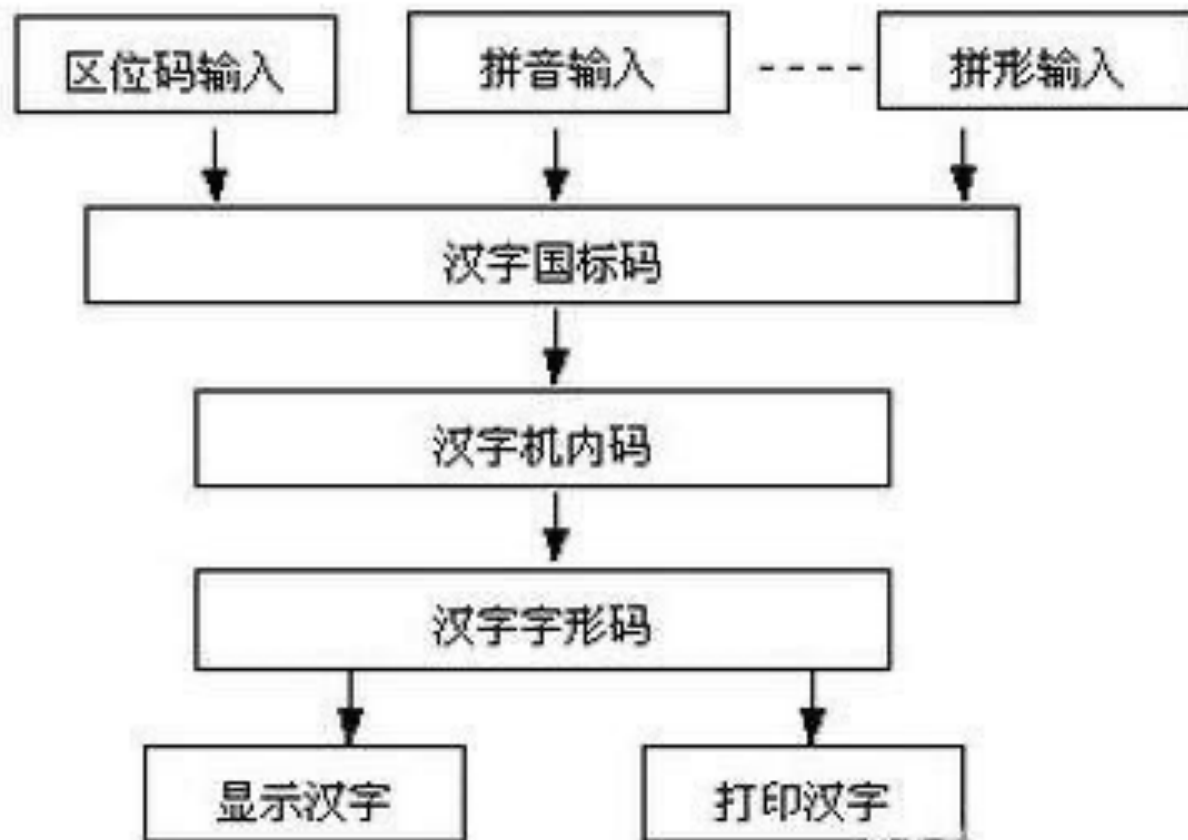
二、非数值数据的表示

GB2312标准中包含的汉字较少，很多生僻字无法表示，很快GB2312标准中没有使用的一些码位也开始用于表示汉字，但后来还是不够用，为此直接**不再要求低字节最高位必须是1**，扩展之后的标准称为**GBK标准（1995）**。该标准兼容了GB2312标准，同时新增了近20000个新的汉字和符号，包括繁体字。后来少数民族文字也被列入该标准中，**新增了4字节的汉字编码，也就是GB18030标准（2005）**。该标准兼容GB2312标准，基本兼容GBK标准，共包括70244个汉字，支持少数民族文字。



二、非数值数据的表示

汉字处理流程





二、非数值数据的表示

③ 汉字机内码（内码）：汉字在计算机内部的编码。为了避免ASCII码和国标码同时使用时产生二义性问题。两个字节，在国际码的每个字节最高位上加1，即

$$\text{汉字机内码} = (\text{汉字国标码})_{16} + 8080\text{H}$$

- 汉字机内码、国标码和区位码三者之间的转换关系：
 - 区位码（十进制）的两个字节分别转换为十六进制后加2020H得到对应的国标码。

$$\text{国标码} = (\text{区位码})_{16} + 2020\text{H}$$

$$\text{机内码} = (\text{国标码})_{16} + 8080\text{H}$$

- 故有：
$$\text{机内码} = (\text{区位码})_{16} + \text{A0A0H}$$



二、非数值数据的表示

【例2.23】以“大”字为例，它的区内码为2083，求国标码和机内码？

解：20是区号，83为位号

(1) 20转换为十六进制数为14，83转换为十六进制数为53，则区内码十六进制表示数为1453H；

(2) 14 53 H

+ 20 20 H

34 73 H

国标码 = 3473H;

(3) 3 4 7 3 H

+ 8 0 8 0 H

11 4 15 3 H

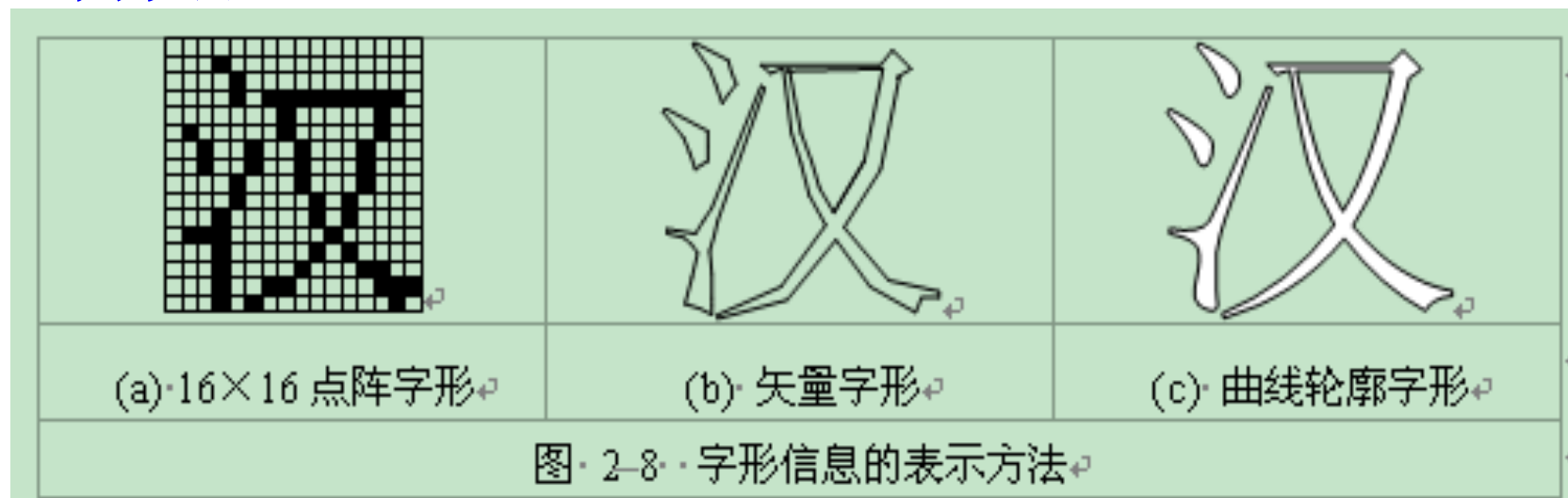
B 4 F 3 H

机内码 = B4F3H;



④汉字的字形码

- 汉字的**输出码**，输出汉字时都采用图形方式，无论汉字的笔画多少，每个汉字都可以写在同样大小的方块中。
- 汉字字型码通常有两种表示方式：**点阵和矢量（轮廓）表示方法**。



汉字地址码是指汉字库中存储汉字字型信息的逻辑地址码。



3 Unicode编码

- 为了统一表示世界各国的文字，**1993年ISO**公布了国际标准**ISO/IEC 10646**，简称**UCS**。为各种文字规定了统一的编码方案，又称为**Unicode**。**2022年9月13日**发布**15.0**版本。
- **Unicode的基本思想**：给每一个字符和符号分配一个永久的、唯一的编码方案。以满足跨语言、跨平台进行文本转换、处理的要求。
- **17个平面**，每个平面有**65536**个码点，可表示的字符超过了**110万**。全世界的语言大概有**20万个**符号。



二、非数值数据的表示

- **Unicode** 只是一个符号集，它只规定了符号的二进制代码，却没有规定这个二进制代码应该如何存储。
- 出现了 **Unicode** 的多种存储方式：**UTF-8, UCS-2, UTF-16, UCS-4, UTF-32**。
- **UTF-8** 是在互联网上使用最广的一种 **Unicode** 的实现方式。
- **UTF-8** 是一种变长的编码方式。它可以使用**1~4**个字节表示一个符号，根据不同的符号而变化字节长度。



二、非数值数据的表示

表 2.8 UTF-8 编码方案

字符码点范围	位数	字节 1	字节 2	字节 3	字节 4
U+0000~U+007F	7	0×××××××			
U+0080~U+07FF	11	110×××××	10××××××		
U+0800~U+FFFF	16	1110××××	10××××××	10××××××	
U+10000~U+10FFFF	21	11110×××	10××××××	10××××××	10××××××

- 1) 对于单字节的符号，字节的第一位设为0，后面7位为这个符号的 Unicode 码。因此对于英语字母，UTF-8 编码和 ASCII 码是相同的。
 - 2) 对于n字节的符号（ $n > 1$ ），第一个字节的前n位都设为1，第n+1位设为0，后面字节的前两位一律设为10。剩下的没有提及的二进制位，全部为这个符号的 Unicode 码。
- 解读 UTF-8 编码非常简单。如果一个字节的第一位是0，则这个字节单独就是一个字符；如果第一位是1，则连续有多少个1，就表示当前字符占用多少个字节。



一、数值数据的表示

1. 数据格式
2. 定点数表示
3. 浮点数表示
4. 计算机中的数据类型

二、非数值数据的表示

1. 字符码
2. 汉字编码
3. Unicode编码

三、数据信息的校验

1. 码距与数据校验
2. 奇偶校验
3. 海明校验
4. 循环冗余校验



数据出错的原因

- 元器件故障
- 存储介质
- 通信过程中的噪声干扰

如何减少或避免数据错误

- 提高计算机系硬件本身的可靠性
- 采取数据检错和校正措施：在每个字中添加一些校验位，用来确定字中出现错误的位置。

经济成本、设计目标



1. 码距与数据校验

- 任何一种编码都有许多码字构成，任何两个相邻码字之间会有n位不同，这就称作是它们之间的海明距离，这些n值中，最小的值就是该种编码的码距。

– 例：BCD编码方案有10个码字：0000，0001，0010，...，1000,1001。

最小码距为1。这种编码没有检错能力，因为当某个合法码字中有1位出错，就变成另一个合法码字了。

- 在纠错理论中，有一个重要公式：

$$d_{\min} \geq t+e+1, \text{ 且 } e > t$$

其中 $d_{\min}$ 为编码码距，t为可纠错的位数，e为可检错的位数。

- $d_{\min} \geq 2$ 的数据校验码具有检错的能力。
- 码距越大，检错和纠错的能力就越强，且检错能力 $\geq$ 纠错能力



三、数据信息的校验

根据信息论原理，码距与校验码的检错和纠错能力的关系为：

- 检错 e 个误码： $d_{\min} \geq e+1$
- 纠错 t 个误码： $d_{\min} \geq 2t+1$
- 纠错 t 个误码，同时检出 e ($e > t$) 个误码： $d_{\min} \geq e+t+1$

码距	检错 (e)	纠错 (t)	检错 (e) 且 纠错 (t)
1	0	0	0, 0
2	1	0	1, 0
3	2	1	1, 1
4	3	1	2, 1
5	4	2	2, 1
6	5	2	3, 2
7	6	3	3, 3



2. 奇偶校验码

最简单且应用广泛的检错码，采用一位校验位：奇校验或偶校验。在数据的首位或末位加上一个校验位。

- 设 $x = x_{n-1}x_{n-2}\dots x_0$ 是一个 n 位字，则奇校验位 $\bar{C}$ 定义为

$$\bar{C} = x_{n-1} \oplus x_{n-2} \oplus \dots \oplus x_0$$

式中 $\oplus$ 代表按位加，表明只有当 x 中包含有奇数个 1 时，才使 $\bar{C} = 1$ ，即 $C = 0$ 。

- 同理，偶校验位 C 定义为

$$C = x_{n-1} \oplus x_{n-2} \oplus \dots \oplus x_0$$

即 x 中包含偶数个 1 时，才使 $C = 0$ 。



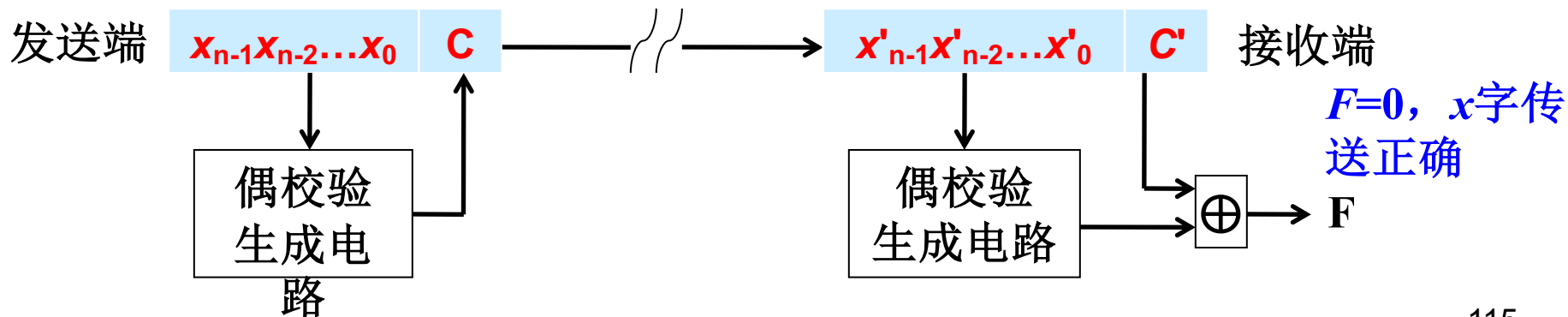
2. 奇偶校验码

校验过程如下

- 在发送端A处，算出校验位C，并与信息码x合并为 $(x_{n-1}x_{n-2}\dots x_0C)$ ，送到接收端B。
- 假设在B点接收到的是 $x'=(x'_{n-1}x'_{n-2}\dots x'_0C')$ ，然后计算

$$F=x'_{n-1}\oplus x'_{n-2}\oplus\dots\oplus x'_0\oplus C'$$

- 偶校验： $F=1$ ，收到信息有错； $F=0$ ， x 字传送正确
- 奇校验： $F=0$ ，收到信息有错； $F=1$ ， x 字传送正确





三、数据信息的校验

- 缺点：
 - 只能检测每个字中所产生的奇数个错误
 - 不具备纠错能力
- 优点
 - 开销小
 - 常用于校验1字节长的数据

通常1字节长的数据编码发生错误时，1位出错的概率较大，两位以上同时错处的概率极小。
- 适用场合：
 - 存储器读写校验
 - 内存的存储过程，发生一位错的概率最大
 - 电路简单、速度快、易于实现
 - 按字节传输中的数据校验：串行传输



【例2.23】 字符 ‘A’ (ASCII码为41H) 的奇校验编码可能是

A. 01000001

B. 11000001 ✓

C. 10000010

D. 10000011 ✓



3. 海明码

- 海明码 (hamming code) 由Richard Hamming于1950年提出。
 - 主要用于存储器数据的校验与纠正。
 - 适用于最有可能发生随机错误的系统
 - 每一位的出错概论相同
 - 每一位与其他位是否出错没有任何关联

海明编码步骤

1. 计算校验位的位数
2. 确认有效信息和校验位的位置
3. 分组
4. 进行奇偶校验，合成海明码



1) 计算校验位的位数

设信息码为 n 位，欲检测1位错误，增添 k 位校验位，组成 $m=n+k$ 位的校验码。为准确对错误定位或指出码字无错，校验位数 k 应满足：

$$2^k \geq n+k+1$$

- k 确定了 2^k 种状态，全0表示无错，余下的组合指示错误位的位置。注：码错误的位置就会有 $n+k$ 个，再加上一个没有错误的可能性，就有 $n+k+1$ 种可能性
- 信息码长度 n 与校验位位数 k 的对应

n	k (最小)
1	2
2~4	3
5~11	4
12~26	5
27~57	6
58~120	7



三、数据信息的校验

2) 确认有效信息和校验位的位置

信息位 (n位) : $D_n D_{n-1} \dots D_2 D_1$
 校验位 (k位) : $P_k P_{k-1} \dots P_2 P_1$ } $H_m H_{m-1} \dots H_2 H_1$ ($m=n+k$)

海明位号

➤ 校验位 P_i 位于位号为 2^{i-1} 的位置, 即

$$P_i = H_j, \quad j = 2^{i-1}, \quad i=1, 2, \dots, k$$

➤ 有效信息则在其余的海明码位置上顺序排列。

例如, 校验位 P_1 位于第 1 位 (H_1), 校验位 P_2 位于第 2 位 (H_2), 校验位 P_3 位于第 4 位 (H_4), 以此类推。 $n=8, k=4$ 的海明码的排列如下

位号	1100	1011	1010	1001	1000	0111	0110	0101	0100	0011	0010	0001
	H_{12}	H_{11}	H_{10}	H_9	H_8	H_7	H_6	H_5	H_4	H_3	H_2	H_1
	D_8	D_7	D_6	D_5	P_4	D_4	D_3	D_2	P_3	D_1	P_2	P_1



3) 分组

分组原则：校验位只参加一组奇偶校验，有效信息则参加至少两组的奇偶校验。

若 $D_i = H_j$ ，则 D_i 参加那些位号之和等于 j 的校验位的分组校验。
如， $D_1(H_3)$ 必须参加 P_1P_2 组的校验， $3=2^1+2^0(i=1,2)$ ， $D_7(H_{11})$ 必须参加 $P_1P_2P_4$ 组的校验($11=2^3+2^1+2^0, i=1,2,4$)。

位号	1100	1011	1010	1001	1000	0111	0110	0101	0100	0011	0010	0001
	H_{12}	H_{11}	H_{10}	H_9	H_8	H_7	H_6	H_5	H_4	H_3	H_2	H_1
	D_8	D_7	D_6	D_5	P_4	D_4	D_3	D_2	P_3	D_1	P_2	P_1
P_4	√	√	√	√	√							
P_3	√					√	√	√	√			
P_2		√	√			√	√			√	√	
P_1		√		√		√		√		√		√



三、数据信息的校验

4) 进行奇偶校验，合成海明码

$n=8$, $k=4$ 的海明码分组偶校验表达式如下：

$$P_4 = D_8 \oplus D_7 \oplus D_6 \oplus D_5$$

$$P_3 = D_8 \oplus D_4 \oplus D_3 \oplus D_2$$

$$P_2 = D_7 \oplus D_6 \oplus D_4 \oplus D_3 \oplus D_1$$

$$P_1 = D_7 \oplus D_5 \oplus D_4 \oplus D_2 \oplus D_1$$

如：有效信息为 11101001，则可以纠错一位的海明码为 1 1 1 0 1 1 0 0 0 1 0 1，带下划线的是校验位 $P_4 P_3 P_2 P_1$ ，分别取值 1001。

位号	1100	1011	1010	1001	1000	0111	0110	0101	0100	0011	0010	0001
	H ₁₂	H ₁₁	H ₁₀	H ₉	H ₈	H ₇	H ₆	H ₅	H ₄	H ₃	H ₂	H ₁
	D ₈	D ₇	D ₆	D ₅	P ₄	D ₄	D ₃	D ₂	P ₃	D ₁	P ₂	P ₁
P ₄	√	√	√	√	√							
P ₃	√					√	√	√	√			
P ₂		√	√			√	√			√	√	
P ₁		√		√		√		√		√		122 √



海明码译码:

按照监督关系式算出指误字 $S_k S_{k-1} \dots S_2 S_1$ 。 $n=8, k=4$ 的海明码分组偶校验的监督表达式:

$$S_4 = P_4 \oplus D_8 \oplus D_7 \oplus D_6 \oplus D_5$$

$$S_3 = P_3 \oplus D_8 \oplus D_4 \oplus D_3 \oplus D_2$$

$$S_2 = P_2 \oplus D_7 \oplus D_6 \oplus D_4 \oplus D_3 \oplus D_1$$

$$S_1 = P_1 \oplus D_7 \oplus D_5 \oplus D_4 \oplus D_2 \oplus D_1$$

$$P_4 = D_8 \oplus D_7 \oplus D_6 \oplus D_5$$

$$P_3 = D_8 \oplus D_4 \oplus D_3 \oplus D_2$$

$$P_2 = D_7 \oplus D_6 \oplus D_4 \oplus D_3 \oplus D_1$$

$$P_1 = D_7 \oplus D_5 \oplus D_4 \oplus D_2 \oplus D_1$$

- 若为全零，则说明各组奇偶性全部无变化，信息正确，将相应的有效信息位析取出来使用；
- 若不全为零，指误字的十进制值，就是出错位的海明位号。

收到的海明码为 1 1 0 0 1 1 0 0 0 1 0 1，计算得到 $S_4 S_3 S_2 S_1=1010$ ，则表明是 H_{10} (D_6) 出错，将 H_{10} 取反即可，正确海明码为 1 1 1 0 1 1 0 0 0 1 0 1。



4. 循环冗余校验码——CRC (Cyclic Redundancy Check)

- **CRC**: 在 n 位信息码后再拼接 k 位校验码, 使整个编码长度为 m 位, 因此这种编码也叫 $(n+k, n)$ 码 (或 (m, n) 码)。
- **CRC码**在数据通信中被广泛采用, 也常用于外存储器的数据校验。编码与校验电路简单。
- **编码原理**: 利用某种数学运算建立有效信息位与校验位之间的约定关系。
- 循环码是一种基于模2运算建立编码规律的校验码。



三、数据信息的校验

- 模2运算规则：二进制运算时不考虑进位和借位。

— 模2加减：按位加或减，用异或逻辑实现。规则如下：

$$0 \pm 0 = 0 \quad 0 \pm 1 = 1 \quad 1 \pm 0 = 1 \quad 1 \pm 1 = 0$$

如 1010	1010	1010
+ 0110	- 0110	+ 1010
1100	1100	0000

$$\begin{array}{r}
 1010 \\
 \times 101 \\
 \hline
 1010 \\
 0000 \\
 1010 \\
 \hline
 100010
 \end{array}
 \left. \vphantom{\begin{array}{r} 1010 \\ \times 101 \\ \hline 1010 \\ 0000 \\ 1010 \\ \hline 100010 \end{array}} \right\} \text{部分积}$$

— 模2乘法：按模2加规则计算部分积之和，不进位。

模2除法：

- 按模2减规则求部分余数，**不借位**。
- 每求一位商，应使**部分余数减少一位**；
- 当部分余数的**首位为1**时，**商取1**；
- 当部分余数的**首位为0**时，**商取0**；
- 当**部分余数的位数小于除数的位数**时，该余数就是最后的余数。

$$\begin{array}{r}
 101 \overline{) 10000} \quad \text{商} \\
 \underline{101} \\
 010 \\
 \underline{000} \\
 100 \\
 \underline{101} \\
 01 \quad \text{余数}
 \end{array}$$



循环冗余校验码的编码规则如下：

- (1) 把待编码的 n 位有效信息表示为多项式 $M(x)$ 。
- (2) 把左移 k 位，以便拼装 k 位余数（即校验位）。
- (3) 选取一个 $k+1$ 位的生成多项式 $G(x)$ ，对 $M(x) \cdot x^k$ 作模2除。
- (4) 把左移 k 位以后的有效信息与余数 $R(x)$ 作模2加减，拼接为CRC码，此时的循环冗余校验码共有 $n+k$ 位。



三、数据信息的校验

- 常用的生成多项式

N	K	码距d	G(x)多项式	G(x)
7	4	3	x^3+x+1	1011
7	4	3	x^3+x^2+1	1101
7	3	4	$x^4+x^3+x^2+1$	11101
7	3	4	x^4+x^2+x+1	10111
15	11	3	x^4+x+1	10011
15	7	5	$x^8+x^7+x^6+x^4+1$	111010001
31	26	3	x^5+x^2+1	100101
31	21	5	$x^{10}+x^9+x^8+x^6+x^5+x^3+1$	11101101001
63	57	3	x^6+x+1	1000011
63	51	5	$x^{12}+x^{10}+x^5+x^4+x^2+1$	1010000110101



三、数据信息的校验

【例2.24】已知二进制编码信息1001，选用的生成码为1011，试为该数据构成CRC码。

解： $M(x)=x^3+1$, $G(x)=x^3+x+1$ ，因为生成码4位，因此 $k=3$ 。

将数据左移3位，得到1001000。进行模2除

$1001000 \div 1011 = 1010$ ，余数为110。

把余数加到二进制信息后面，得到1001110，即CRC码。

$$\begin{array}{r} 1010 \\ 1011 \overline{) 1001000} \\ \underline{1011} \\ 0100 \\ \underline{0000} \\ 1000 \\ \underline{1011} \\ 0110 \\ \underline{0000} \\ 110 \end{array}$$

$$\begin{array}{r} 1010 \\ 1011 \overline{) 1001110} \\ \underline{1011} \\ 0101 \\ \underline{0000} \\ 1011 \\ \underline{1011} \\ 0000 \\ \underline{0000} \\ 000 \end{array}$$

$$\begin{array}{r} 1011 \\ 1011 \overline{) 1001110} \\ \underline{1011} \\ 0111 \\ \underline{0000} \\ 1111 \\ \underline{1011} \\ 1000 \\ \underline{1011} \\ 011 \end{array}$$



三、数据信息的校验

$G(X) = 1011_2$	编码举例1		编码举例2		余数	出错位置
	数 据 位 6 5 4 3	校 验 位 2 1 0	数 据 位 6 5 4 3	校 验 位 2 1 0		
正确	1 0 0 1	1 1 0	1 1 0 0	0 1 0	0 0 0	无
错误	1 0 0 1	1 1 1	1 1 0 0	0 1 1	0 0 1	0
	1 0 0 1	1 0 0	1 1 0 0	0 0 0	0 1 0	1
	1 0 0 1	0 1 0	1 1 0 0	1 1 0	1 0 0	2
	1 0 0 0	1 1 0	1 1 0 1	0 1 0	0 1 1	3
	1 0 1 1	1 1 0	1 1 1 0	0 1 0	1 1 0	4
	1 1 0 1	1 1 0	1 0 0 0	0 1 0	1 1 1	5
	0 0 0 1	1 1 0	0 1 0 0	0 1 0	1 0 1	6

(7, 4)循环码编码、余数与出错位置的关系



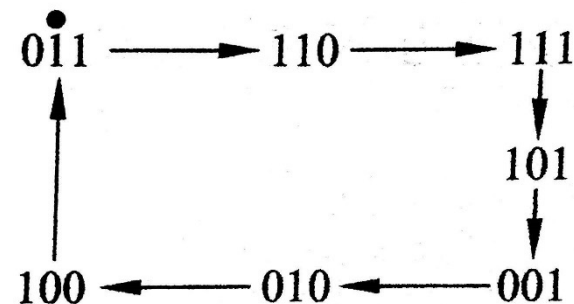
三、数据信息的校验

CRC的纠错

方法一：通过查表确定CRC码的出错位，对出错位求反即可纠正。

方法二：CRC码出错时，则用 $G(x)$ 做模2除得到一个不为0的余数。若对余数补0继续除下去，将发现各次余数将出现循环。如左起第 n 位出错，则余数除 $n-1$ 次后得到101。

- 如果把出错码从左往右计算位数，则左起第1位出错时余数为101；第2位出错时余数为111，把111继续用1011除，除一次后即得到101；第3位出错，则把余数连除两次，即得到101。



从这一规律可得出一个简单的纠错方法：

- (1) 余数为0时，表示无错。
- (2) 余数为101时，左起第一位出错。
- (3) 余数非0又非101时，继续模2除法。设除 $n-1$ 次得到余数101，则从左起 n 位出错。



THE END !

THANKS